

MODULE 2: CLINICAL BIOSTATISTICS FOR KIDNEY DISEASE RESEARCH

Darya Y

2026-06-14

```
options(width = 70)
if (!requireNamespace("formatR", quietly = TRUE)) install.packages("formatR")
knitr::opts_chunk$set(
  tidy = TRUE,
  tidy.opts = list(width.cutoff = 65),
  comment = NA,
  cache = TRUE
)

# =====
# MODULE 2: CLINICAL BIOSTATISTICS FOR KIDNEY DISEASE RESEARCH
# =====
#
# INTRODUCTION: WHAT IS CLINICAL BIOSTATISTICS IN THIS CONTEXT?

# A kidney disease research collaboration.
# The data looks like this:
#   - Patients with Chronic Kidney Disease (CKD), recruited at a clinic
#   - Measurements taken at multiple time points (e.g., baseline, 6 months, 12 months)
#   - Lab values: eGFR (estimated Glomerular Filtration Rate) - the gold standard
#     measure of kidney function. Normal is ~90+. Below 60 = CKD. Below 15 = failure.
#   - Immune phenotyping from flow cytometry: monocyte scores, T cell scores
#     These tell us about the inflammatory state of the patient.
#
# THE CORE QUESTION:
#   "Can we predict which CKD patients will progress to kidney failure based on
#     their immune cell profiles measured early in the disease course?"
#
# WHY THIS MATTERS:
#   - If monocyte activation (a marker of chronic inflammation) predicts faster
#     eGFR decline, clinicians could intensify immunosuppression earlier
#   - This is a real research question - innate immune cells are implicated in
#     CKD progression through tubulointerstitial fibrosis
#
# STATISTICAL CHALLENGES IN THIS DATA:
#   1. REPEATED MEASURES: Same patient at 3 time points → not independent
#   2. MISSING DATA: Patients drop out (often the sickest ones - informative censoring)
#   3. CONFOUNDING: Age, sex, diabetes status all affect eGFR
#   4. MULTIPLE TESTING: If we test 50 immune markers, ~2.5 will be significant by chance
#
# This module teaches how to handle all four challenges rigorously.
# =====
```

```

# =====
# SECTION 0: PACKAGE INSTALLATION & LOADING
# =====
# Check for each package before attempting install.
# suppressMessages() reduces noise during loading - useful in production scripts.

required_packages <- c("dplyr", "ggplot2", "lme4", "lmerTest", "survival",
                      "broom", "broom.mixed", "tidyr", "tidyverse", "ggpubr",
                      "ggfortify", "patchwork", "scales", "RColorBrewer",
                      "survminer", "mediation", "simr")

for (pkg in required_packages) {
  if (!requireNamespace(pkg, quietly = TRUE)) {
    message(paste("Installing:", pkg))
    install.packages(pkg, repos = "https://cloud.r-project.org", quiet = TRUE)
  }
}

suppressMessages({
  library(dplyr)
  library(ggplot2)
  library(lme4)
  library(lmerTest) # Adds p-values to lme4 output via Satterthwaite approximation
  library(survival)
  library(broom)
  library(broom.mixed) # tidy() for mixed effects models
  library(tidyr)
  library(patchwork) # Combine ggplot panels
  library(scales)
  library(RColorBrewer)
  library(tidyverse)
  library(ggpubr)
  library(ggfortify)
  library(survminer)
  library(mediation)
  library(simr)
})

# Create output directory for plots
plot_dir <- "module2_plots"
dir.create(plot_dir, showWarnings = FALSE, recursive = TRUE)

cat("=== MODULE 2: Clinical Biostatistics ===\n")

=== MODULE 2: Clinical Biostatistics ===
cat("Output plots will be saved to:", plot_dir, "\n\n")

```

Output plots will be saved to: module2_plots

```

# =====
# SECTION 1: SIMULATE REALISTIC PATIENT DATA
# =====
# WHY SIMULATE? In research, we often need to:
# - Test the analysis pipeline before real data arrives
# - Demonstrate statistical power (simulate → analyze → check results)

```

```

#
# KEY DESIGN DECISIONS IN THIS SIMULATION:
# - 120 patients: 80 CKD + 40 Healthy controls (realistic 2:1 ratio)
# - 3 time points: baseline (0), 6 months, 12 months
# - eGFR: CKD patients decline ~2-3 mL/min/1.73m2 per 6 months on average
#   (published literature on CKD progression rates)
# - Immune scores: monocyte_score elevated in CKD, and HIGHER scores correlate
#   with FASTER decline (the hypothesis)
# - Random effects: each patient has their own baseline eGFR and their own
#   rate of decline (some progress fast, some slow - biological variability)

set.seed(42) # For reproducibility

n_ckd      <- 80 # CKD patients
n_healthy  <- 40 # Healthy controls
n_total    <- n_ckd + n_healthy
time_points <- c(0, 6, 12) # months

# --- Simulate patient-level (baseline) characteristics ---
# Fixed properties of each patient, measured once

patient_data <- data.frame(
  patient_id = paste0("PT", sprintf("%03d", 1:n_total)),
  condition  = c(rep("CKD", n_ckd), rep("Healthy", n_healthy)),

  # Age: CKD patients tend to be older (mean 62) vs healthy controls (mean 48)
  # rnorm(n, mean, sd) - normal distribution
  age = c(round(rnorm(n_ckd, mean = 62, sd = 12)),
           round(rnorm(n_healthy, mean = 48, sd = 15))),

  # Sex: roughly 55% male in CKD cohorts (CKD is slightly male-predominant)
  sex = c(sample(c("Male", "Female"), n_ckd, replace = TRUE, prob = c(0.55, 0.45)),
           sample(c("Male", "Female"), n_healthy,
                  replace = TRUE, prob = c(0.50, 0.50))),

  # Baseline eGFR:
  # CKD: mean 52, SD 12 (Stage 3 CKD range: 30-59)
  # Healthy: mean 88, SD 10 (normal range: 60-120)
  # pmax(25, ...) floors eGFR at 25; pmin(120, ...) caps it at 120
  eGFR_baseline = c(pmax(25, rnorm(n_ckd, mean = 52, sd = 12)),
                     pmin(120, pmax(60, rnorm(n_healthy, mean = 88, sd = 10)))),

  # Monocyte score from flow cytometry (arbitrary units, higher = more activated)
  # CKD patients have chronically activated monocytes
  # (NF-κB signalling, IL-6 production)
  monocyte_score = c(rnorm(n_ckd, mean = 0.65, sd = 0.18),
                     rnorm(n_healthy, mean = 0.30, sd = 0.12)),

  # T cell score: CKD also affects T cell exhaustion
  t_cell_score = c(rnorm(n_ckd, mean = 0.45, sd = 0.15),
                   rnorm(n_healthy, mean = 0.55, sd = 0.12)),

  # Individual rate of eGFR decline (random effect):
  # Each patient has their own "speed" of disease progression
  # Mean decline: -2.5 mL/min per 6 months for CKD; ~0 for healthy
  # SD 1.5: some patients progress much faster (rapid progressors)

```

```

decline_rate = c(rnorm(n_ckd, mean = -2.5, sd = 1.5),
                 rnorm(n_healthy, mean = 0, sd = 0.3)),

# Keep strings as characters, not factors
stringsAsFactors = FALSE
)

# Ensure monocyte scores are bounded (0, 1) - these represent proportions
patient_data$monocyte_score <- pmin(1, pmax(0, patient_data$monocyte_score))
patient_data$t_cell_score   <- pmin(1, pmax(0, patient_data$t_cell_score))

# --- Expand to long format (one row per patient per time point) ---
# Each patient contributes 3 rows (one per time point)

longitudinal_data <- patient_data %>%
  # crossing() creates all combinations of patient × time (Cartesian product)
  tidyr::crossing(time_months = time_points) %>%
  mutate(
    # eGFR at each time point:
    # baseline + (individual decline rate × time) + measurement noise
    # The noise term (rnorm) represents lab measurement error (~2 mL/min SD)
    eGFR = eGFR_baseline + decline_rate * (time_months / 6) +
          rnorm(n(), mean = 0, sd = 2),

    # Floor eGFR at 10 (dialysis threshold - patients are removed from study)
    eGFR = pmax(10, eGFR),

    # Dynamic monocyte score and T-cell score over time
    monocyte_score = case_when(
      condition == "CKD" ~ monocyte_score + 0.04 * (time_months / 6) +
        rnorm(n(), mean = 0, sd = 0.04),
      condition == "Healthy" ~ monocyte_score + rnorm(n(), mean = 0, sd = 0.02)
    ),
    # Ensure scores remain physically realistic proportions between 0 and 1
    monocyte_score = pmin(1, pmax(0, monocyte_score)),

    t_cell_score = case_when(
      condition == "CKD" ~ t_cell_score + 0.02 * (time_months / 6) +
        rnorm(n(), mean = 0, sd = 0.03),
      condition == "Healthy" ~ t_cell_score + rnorm(n(), mean = 0, sd = 0.02)
    ),
    # Ensure scores remain physically realistic proportions between 0 and 1
    t_cell_score = pmin(1, pmax(0, t_cell_score)),

    # Time as factor for some analyses (categorical: "Baseline", "6mo", "12mo")
    time_factor = factor(time_months,
                        levels = c(0, 6, 12),
                        labels = c("Baseline", "6 months", "12 months"))
  ) %>%
  # Convert condition and sex to factors (important for regression contrast coding)
  mutate(
    condition = factor(condition, levels = c("Healthy", "CKD")), # ref = Healthy
    sex = factor(sex, levels = c("Female", "Male")) # ref = Female
  )

cat("Longitudinal dataset created:\n")

```

Longitudinal dataset created:

```
cat(" Rows:", nrow(longitudinal_data), "(should be", n_total * 3, ")\n")
```

Rows: 360 (should be 360)

```
cat(" Patients:", n_distinct(longitudinal_data$patient_id), "\n")
```

Patients: 120

```
cat(" Columns:", paste(names(longitudinal_data), collapse = ", "), "\n\n")
```

Columns: patient_id, condition, age, sex, eGFR_baseline, monocyte_score, t_cell_score, decline_rate, time

```
# =====  
# SECTION 2: DESCRIPTIVE STATISTICS (TABLE 1)  
# =====  
# Every clinical paper has a "Table 1" - baseline characteristics of the cohort; showing:  
# 1. Continuous variables: mean  $\pm$  SD (or median [IQR] if non-normal)  
# 2. Categorical variables: n (%)  
# 3. P-value for group differences (t-test for continuous, chi-squared for categorical)  
#  
# WHY TABLE 1 MATTERS:  
# If CKD patients are much older than controls, and eGFR naturally declines with age,  
# we need to adjust for age in the models. Table 1 reveals potential confounders.  
  
cat("=== TABLE 1: BASELINE CHARACTERISTICS ===\n\n")
```

=== TABLE 1: BASELINE CHARACTERISTICS ===

```
# Use only baseline time point for Table 1  
baseline_data <- longitudinal_data %>%  
  filter(time_months == 0)  
  
# --- Continuous variables: mean  $\pm$  SD by condition ---  
# summarise() aggregates data as needed  
table1_continuous <- baseline_data %>%  
  group_by(condition) %>%  
  summarise(  
    n = n(),  
    Age_mean = round(mean(age), 1),  
    Age_sd = round(sd(age), 1),  
    eGFR_mean = round(mean(eGFR), 1),  
    eGFR_sd = round(sd(eGFR), 1),  
    Mono_mean = round(mean(monocyte_score), 3),  
    Mono_sd = round(sd(monocyte_score), 3),  
    Tcell_mean = round(mean(t_cell_score), 3),  
    Tcell_sd = round(sd(t_cell_score), 3),  
    .groups = "drop"  
  )  
  
cat("Continuous Variables (Mean  $\pm$  SD):\n")
```

Continuous Variables (Mean $\pm$ SD):

```
print(table1_continuous)
```

A tibble: 2 x 10

	condition	n	Age_mean	Age_sd	eGFR_mean	eGFR_sd	Mono_mean	Mono_sd
	<fct>	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	Healthy	40	48.8	14.6	89	11.6	0.306	0.119

```
2 CKD          80      62.2  12.9      51.5   11.4      0.636  0.157
# i 2 more variables: Tcell_mean <dbl>, Tcell_sd <dbl>
```

```
# --- Sex distribution by condition ---
table1_sex <- baseline_data %>%
  count(condition, sex) %>%
  group_by(condition) %>%
  mutate(pct = round(100 * n / sum(n), 1)) %>%
  ungroup()

cat("\nSex Distribution:\n")
```

Sex Distribution:

```
print(table1_sex)
```

```
# A tibble: 4 x 4
  condition sex      n  pct
  <fct>      <fct> <int> <dbl>
1 Healthy  Female     23  57.5
2 Healthy  Male      17  42.5
3 CKD      Female     24   30
4 CKD      Male     56  70
```

```
# --- Statistical tests for group differences ---
# t.test() for continuous variables (tests if means differ between CKD vs Healthy)
# Use var.equal = FALSE (Welch's t-test) - when group sizes/variances differ
```

```
ckd_base <- baseline_data %>% filter(condition == "CKD")
healthy_base <- baseline_data %>% filter(condition == "Healthy")

age_test <- t.test(ckd_base$age, healthy_base$age, var.equal = FALSE)
egfr_test <- t.test(ckd_base$egfr, healthy_base$egfr, var.equal = FALSE)
mono_test <- t.test(ckd_base$monocyte_score,
  healthy_base$monocyte_score, var.equal = FALSE)
```

```
# chisq.test() for categorical variables
sex_table <- table(baseline_data$condition, baseline_data$sex)
sex_test <- chisq.test(sex_table)

cat("\nGroup Comparison P-values:\n")
```

Group Comparison P-values:

```
cat(sprintf(" Age:                p = %.4f\n", age_test$p.value))
```

Age: p = 0.0000

```
cat(sprintf(" Baseline eGFR:  p = %.4f\n", egfr_test$p.value))
```

Baseline eGFR: p = 0.0000

```
cat(sprintf(" Monocyte score: p = %.4f\n", mono_test$p.value))
```

Monocyte score: p = 0.0000

```
cat(sprintf(" Sex (chi-sq):    p = %.4f\n", sex_test$p.value))
```

Sex (chi-sq): p = 0.0067

```

# INTERPRETATION GUIDE:
# - eGFR p-value should be highly significant. By design, the simulated CKD cohort
#   has a lower baseline kidney function.
# - Monocyte score p-value should be significant (elevated in CKD by design)
# - Age p-value should be significant (we intentionally simulated older CKD
#   patients to reflect real-world epidemiological data.)
# - Sex p-value would be significant because we intentionally simulated a higher
#   proportion of males in the CKD group to reflect real-world epidemiological data.

# =====
# SECTION 3: VISUALISATION
# =====
# Visualise the data BEFORE running statistical models.
# Plots reveal: outliers, non-linearity, missing data patterns, batch effects.
#
# ggplot2 GRAMMAR REMINDER:
#   ggplot(data, aes(x=..., y=..., color=...)) +      # map variables to aesthetics
#   geom_point() +                                    # choose a geometry
#   facet_wrap(~ variable) +                          # panel by variable
#   theme_minimal() +                                # clean theme
#   labs(title=..., x=..., y=...)                    # labels

cat("\n=== SECTION 3: VISUALISATION ===\n")

```

```

=== SECTION 3: VISUALISATION ===

```

```

# --- PLOT 1: eGFR Trajectories Over Time ---
# Goal: Show individual patient trajectories + group mean trend
# Spaghetti plot - each line = one patient
# CKD patients decline while Healthy stay stable

# Define colour palette - using CKD-appropriate colours
condition_colors <- c("Healthy" = "#2196F3",      # Blue for healthy
                      "CKD"      = "#F96006")    # Red for disease

# Individual trajectories (thin, semi-transparent lines)
p_egfr_traj <- ggplot(longitudinal_data,
                      aes(x = time_months, y = eGFR,
                          group = patient_id, color = condition)) +

# Individual lines
  geom_line(alpha = 0.15, linewidth = 0.4) +

# Group mean lines
# stat_summary computes mean at each time point and draws a line
  stat_summary(aes(group = condition),
               fun = mean, geom = "line",
               linewidth = 1, linetype = "solid") +

# Add mean points at each time point
  stat_summary(aes(group = condition),
               fun = mean, geom = "point",
               size = 3, shape = 16) +

# Add shaded confidence band around group mean ( $\pm 1$  SE)

```

```

stat_summary(aes(group = condition, fill = condition),
             fun.data = mean_se, geom = "ribbon",
             alpha = 0.2, color = NA) +

# Reference line: eGFR = 30 is the threshold for Stage 4 CKD (high risk)
geom_hline(yintercept = 30, linetype = "dashed", color = "#F01006", alpha = 0.6) +
annotate("text", x = 10.5, y = 32, label = "eGFR=30 (Stage 4 CKD)",
        size = 3, color = "#F01006", hjust = 1) +

scale_color_manual(values = condition_colors) +
scale_fill_manual(values = condition_colors) +
scale_x_continuous(breaks = c(0, 6, 12),
                  labels = c("Baseline", "6 months", "12 months")) +

labs(
  title      = "eGFR Trajectories Over 12 Months",
  subtitle   = "Individual lines (faint) with group mean ± SE (thick + shading)",
  x          = "Time Point",
  y          = "eGFR (mL/min/1.73m²)",
  color      = "Condition",
  fill       = "Condition",
  caption    = "Dashed red line = Stage 4 CKD threshold (eGFR < 30)"
) +
theme_minimal(base_size = 8) +
theme(
  plot.title      = element_text(face = "bold"),
  legend.position = "top",
  panel.grid.minor = element_blank()
)

ggsave(file.path(plot_dir, "01_eGFR_trajectories.png"),
       p_egfr_traj, width = 8, height = 6, dpi = 300)
cat("Saved: 01_eGFR_trajectories.png\n")

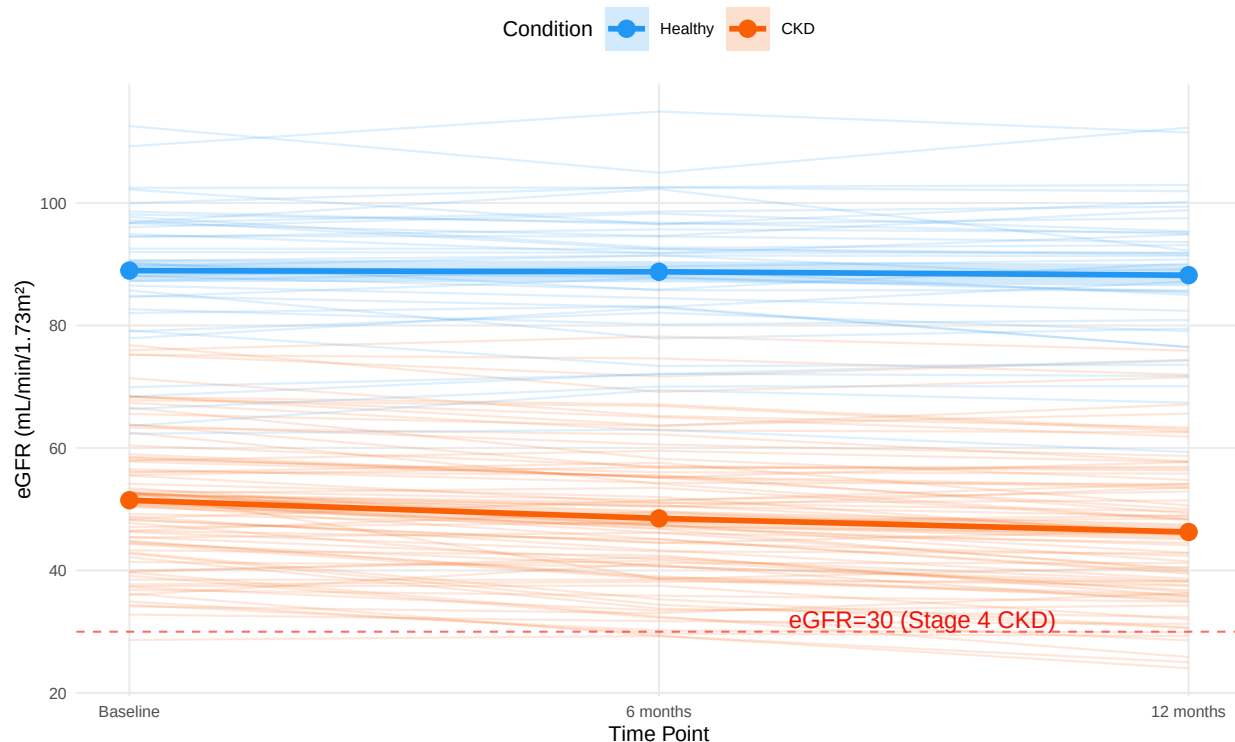
```

Saved: 01_eGFR_trajectories.png

```
print(p_egfr_traj)
```


eGFR Trajectories Over 12 Months

Individual lines (faint) with group mean $\pm$ SE (thick + shading)



Dashed red line = Stage 4 CKD threshold (eGFR < 30)

```
# --- PLOT 2: Immune Score Distributions by Condition ---
# Violin plot - shows the full distribution shape
# Useful for spotting bimodality (two subpopulations within CKD)

# Define exactly which 'condition's to compare (required for adding the automated significance)
comparison <- list( c("Healthy", "CKD") )

p_mono_violin <- ggplot(baseline_data,
                        aes(x = condition, y = monocyte_score, fill = condition)) +

  # Violin: shows the distribution density
  geom_violin(trim = FALSE, alpha = 0.7) +

  # Boxplot inside the violin - shows median and IQR
  # width=0.1 makes it narrow so it sits inside the violin
  geom_boxplot(width = 0.1, fill = "white", outlier.shape = NA) +

  # Individual points: jittered to avoid overplotting
  geom_jitter(width = 0.05, size = 1, alpha = 0.4) +

  scale_fill_manual(values = condition_colors) +

  # Add the automated statistics
  stat_compare_means(
    comparisons = comparison,
    method = "t.test",      # Or "wilcox.test" for non-parametric
    label = "p.format"     # Or "p.signif" for *** stars
  ) +
```

```

labs(
  title     = "Monocyte Activation Score by Condition",
  subtitle  = "Baseline measurements only",
  x         = "Condition",
  y         = "Monocyte Score (AU)",
  fill      = "Condition"
) +
theme_minimal(base_size = 8) +
theme(plot.title = element_text(face = "bold"),
      legend.position = "none")

p_tcell_violin <- ggplot(baseline_data,
                        aes(x = condition, y = t_cell_score, fill = condition)) +
  geom_violin(trim = FALSE, alpha = 0.7) +
  geom_boxplot(width = 0.1, fill = "white", outlier.shape = NA) +
  geom_jitter(width = 0.05, size = 1, alpha = 0.4) +
  scale_fill_manual(values = condition_colors) +
  labs(
    title     = "T Cell Score by Condition",
    subtitle  = "Baseline measurements only",
    x         = "Condition",
    y         = "T Cell Score (AU)",
    fill      = "Condition"
  ) +
  theme_minimal(base_size = 8) +
  theme(plot.title = element_text(face = "bold"),
        legend.position = "none")

# Combine using patchwork
p_immune_combined <- p_mono_violin | p_tcell_violin
ggsave(file.path(plot_dir, "02_immune_scores_violin.png"),
       p_immune_combined, width = 10, height = 6, dpi = 300)
cat("Saved: 02_immune_scores_violin.png\n")

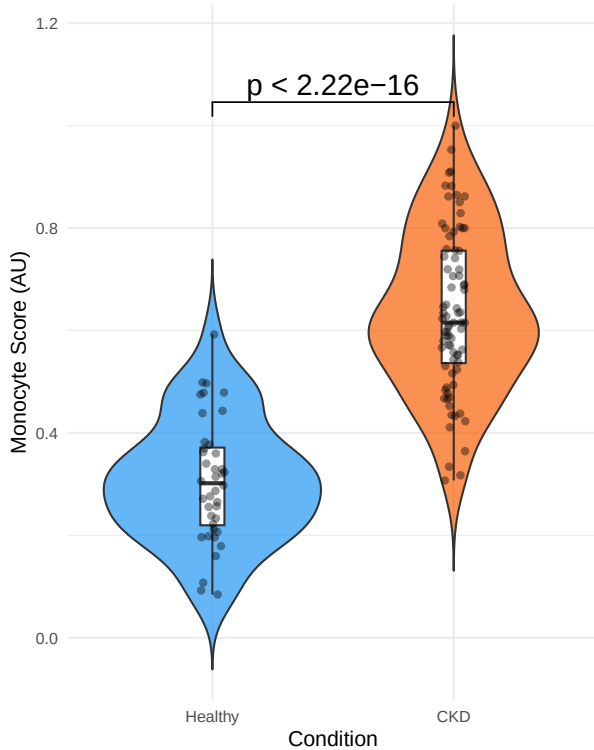
```

Saved: 02_immune_scores_violin.png

```
print(p_immune_combined)
```

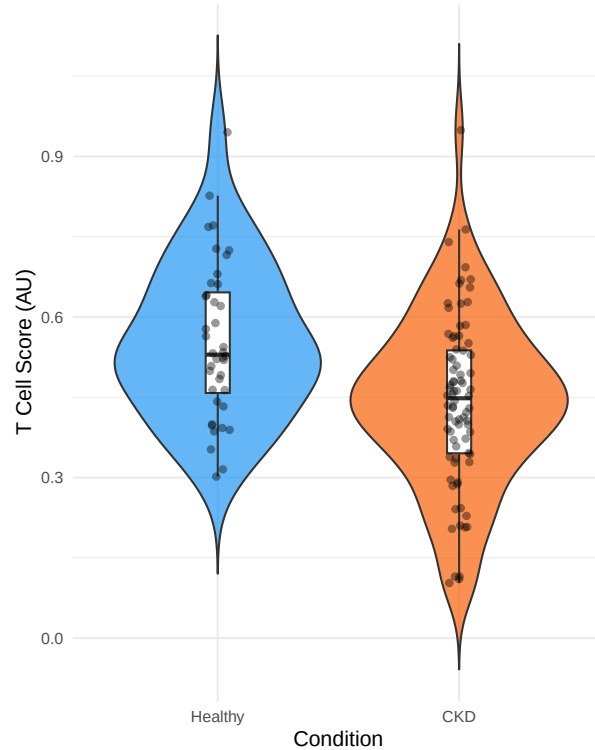
Monocyte Activation Score by Condition

Baseline measurements only



T Cell Score by Condition

Baseline measurements only



```
# --- PLOT 3: Scatter - Monocyte Score vs eGFR change ---
# Does higher monocyte activation at baseline predict more eGFR decline?
# This is the core biological hypothesis visualised directly.

# Calculate eGFR change from baseline to 12 months for each CKD patient.
# monocyte_score is extracted separately at t=0 to avoid pivot_wider
# producing 3 rows per patient (one per time point).
mono_baseline_scatter <- longitudinal_data %>%
  filter(condition == "CKD", time_months == 0) %>%
  dplyr::select(patient_id, monocyte_score)

egfr_change <- longitudinal_data %>%
  filter(condition == "CKD") %>%
  dplyr::select(patient_id, time_months, eGFR) %>%
  pivot_wider(names_from = time_months, values_from = eGFR,
    names_prefix = "eGFR_t") %>%
  mutate(eGFR_change = `eGFR_t12` - `eGFR_t0`) %>% # negative = decline
  left_join(mono_baseline_scatter, by = "patient_id") %>%
  filter(!is.na(eGFR_change), !is.na(monocyte_score))

p_scatter <- ggplot(egfr_change,
  aes(x = monocyte_score, y = eGFR_change)) +
  geom_point(alpha = 0.6, color = "#F96006", size = 2) +
  geom_smooth(method = "lm", se = TRUE, color = "#F01006", fill = "pink") +

  # Automatically calculate and plot the correlation (r) AND its significance (p)
  stat_cor(method = "pearson", label.x.npc = "left", label.y.npc = "bottom") +

  geom_hline(yintercept = 0, linetype = "dashed", color = "grey50") +
```

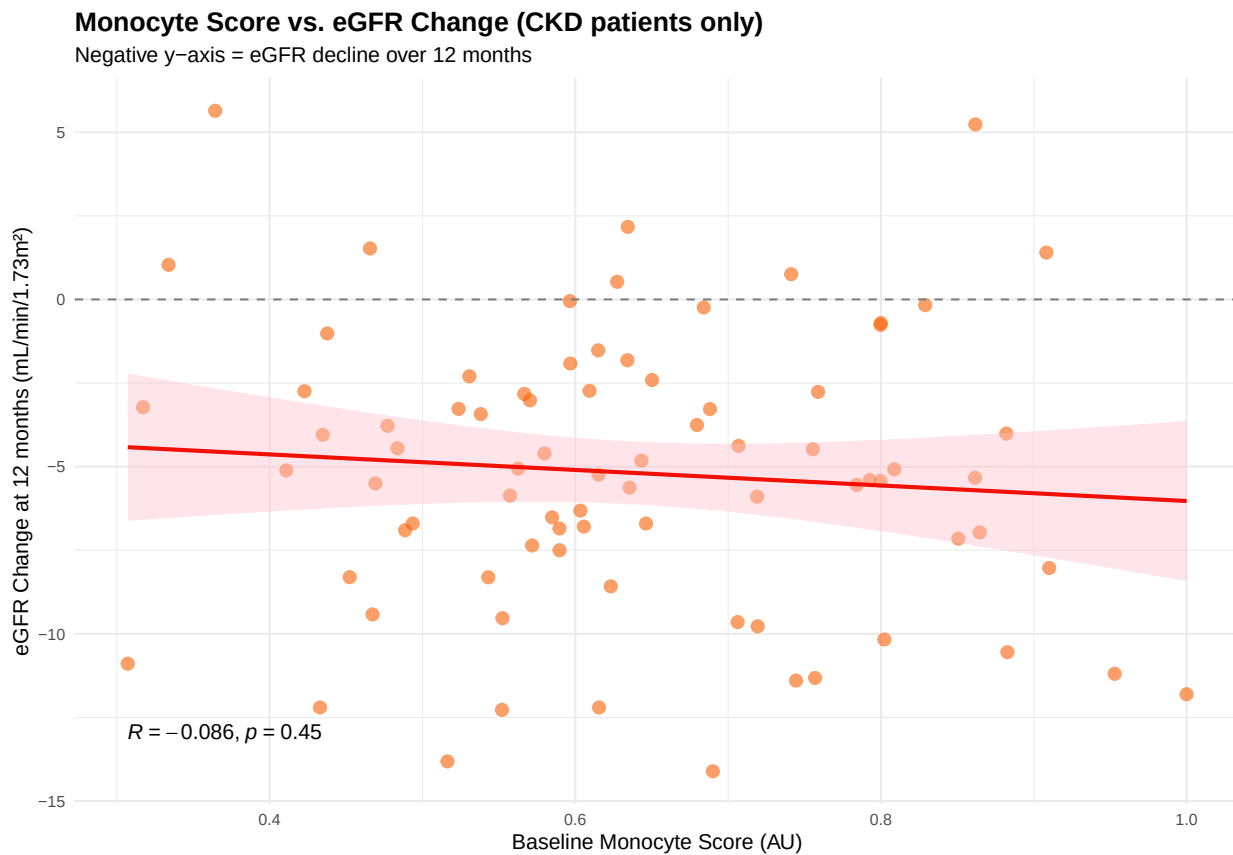
```
labs(
  title = "Monocyte Score vs. eGFR Change (CKD patients only)",
  subtitle = "Negative y-axis = eGFR decline over 12 months",
  x = "Baseline Monocyte Score (AU)",
  y = "eGFR Change at 12 months (mL/min/1.73m²)"
) +
theme_minimal(base_size = 8) +
theme(plot.title = element_text(face = "bold"))

ggsave(file.path(plot_dir, "03_monocyte_vs_egfr_change.png"),
  p_scatter, width = 7, height = 6, dpi = 300)
```

```
`geom_smooth()` using formula = 'y ~ x'
cat("Saved: 03_monocyte_vs_egfr_change.png\n")
```

```
Saved: 03_monocyte_vs_egfr_change.png
print(p_scatter)
```

```
`geom_smooth()` using formula = 'y ~ x'
```



```
# =====
# SECTION 4: MIXED EFFECTS MODELS (lme4)
# =====
#
# THE FUNDAMENTAL PROBLEM WITH REPEATED MEASURES:
#
# Standard linear regression assumes observations are INDEPENDENT.
# But in longitudinal data, three measurements from the same patient
# are NOT independent. PT001 at months 0, 6, 12 shares the same
```

```

# genetics, lifestyle, disease severity - violating regression assumptions.
#
# If we use plain lm() ignoring this, we will:
# 1. Underestimate standard errors (→ false positives)
# 2. Not correctly separate "within-patient" from "between-patient" effects
#
# MIXED EFFECTS MODELS solve this by:
# - FIXED EFFECTS: population-average effects
#   (what we care about: does CKD affect eGFR?)
# - RANDOM EFFECTS: patient-specific deviations
#   (each patient has their own "offset")
#   The random effects absorb the correlation within patients.
#   They represent the biological noise or individual deviations from
#   the grand average.
#   By splitting the model into these two parts, the fixed effects become
#   incredibly precise because they are no longer confused by the individual
#   biological quirks of the specific patients in your dataset.
#    $y = 0 + 1x + u0 + u1x$  where  $x$  terms are the Fixed Effects and
#    $u$  terms are the Random Effects
#
# MODEL NOTATION: lmer(eGFR ~ time * condition + (1|patient_id), data=...)
# - eGFR: outcome variable
# - time * condition: fixed effects (main effects + interaction)
#   The interaction tests: "does the effect of TIME differ by CONDITION?"
#   i.e., "do CKD patients decline faster than Healthy?"
# - (1|patient_id): random intercept per patient
#   Each patient is allowed their own baseline eGFR level
#   This is the minimal random effects structure for repeated measures
#
# WHY NOT ALSO (time|patient_id)?
# A random slope for time would additionally allow each patient their own
# rate of change. This is more flexible but requires more data to estimate.
# For 3 time points, a random intercept is usually sufficient.

cat("\n=== SECTION 4: MIXED EFFECTS MODELS ===\n\n")

```

```

=== SECTION 4: MIXED EFFECTS MODELS ===

```

```

# --- MODEL 1: Does CKD condition affect eGFR trajectory? ---
# This is confirmatory: we know CKD patients have lower eGFR.
# But this tests whether the RATE OF CHANGE differs between groups.
#
# Fixed effects interpretation:
# Intercept: mean eGFR in Healthy group at time=0 (baseline)
# time_months: slope of eGFR vs time in Healthy group (should be ~0)
# conditionCKD: how much lower eGFR is in CKD vs Healthy at baseline
# time_months:conditionCKD: INTERACTION = extra slope in CKD vs Healthy
# A negative value means CKD declines faster over time

```

```

cat("--- Model 1: eGFR ~ time × condition ---\n")

```

```

--- Model 1: eGFR ~ time × condition ---

```

```

model1 <- lmer(
  eGFR ~ time_months * condition + (1 | patient_id),
  data = longitudinal_data,
  REML = TRUE # REML = Restricted Maximum Likelihood
)

```

```

        # Use REML for estimating random effects (less biased than ML)
        # Use ML (REML=FALSE) when comparing fixed effects
    )

cat("\nModel 1 Summary:\n")

```

Model 1 Summary:

```
print(summary(model1))
```

```

Linear mixed model fit by REML. t-tests use Satterthwaite's method [
lmerModLmerTest]

```

```
Formula: eGFR ~ time_months * condition + (1 | patient_id)
```

```
Data: longitudinal_data
```

```
REML criterion at convergence: 2163.7
```

Scaled residuals:

```

      Min       1Q   Median       3Q      Max
-2.55523 -0.59399 -0.02542  0.54223  2.53763

```

Random effects:

```

Groups      Name      Variance Std.Dev.
patient_id (Intercept) 127.815  11.31
Residual              5.907   2.43

```

```
Number of obs: 360, groups: patient_id, 120
```

Fixed effects:

```

              Estimate Std. Error      df t value
(Intercept)    89.02532    1.82166 123.39985  48.871
time_months    -0.06283    0.04529 238.00000  -1.387
conditionCKD   -37.68601    2.23106 123.39986 -16.891
time_months:conditionCKD -0.36906    0.05547 238.00000  -6.654

```

```
Pr(>|t|)
```

```

(Intercept)    < 2e-16 ***
time_months      0.167
conditionCKD    < 2e-16 ***
time_months:conditionCKD 1.95e-10 ***
---

```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Correlation of Fixed Effects:

```

      (Intr) tm_mnt cndCKD
time_months -0.149
conditinCKD -0.816  0.122
tm_mnth:CKD  0.122 -0.816 -0.149

```

```

patient_var <- as.data.frame(VarCorr(model1))$vcov[1]
residual_var <- as.data.frame(VarCorr(model1))$vcov[2]
icc <- patient_var / (patient_var + residual_var)
cat(sprintf("\nModel 1 - Intraclass Correlation (ICC): %.3f\n", icc))

```

Model 1 - Intraclass Correlation (ICC): 0.956

```

cat(sprintf(
  "Interpretation: %.1f%% of variance is between-patient\n",

```

```
icc * 100))
```

Interpretation: 95.6% of variance is between-patient

```
cat("ICC > 0.5 = substantial clustering => mixed model is essential!\n\n")
```

ICC > 0.5 = substantial clustering => mixed model is essential!

```
# --- Understanding the model output ---
```

```
# Random effects section:
```

```
# Groups: patient_id - this tells us the variance between patients
```

```
# Residual: variance within patients (unexplained noise)
```

```
# ICC (Intraclass Correlation) = patient variance / (patient + residual variance)
```

```
# ICC tells us: "what fraction of total variance is due to patient identity?"
```

```
# High ICC (>0.5) → patients are very different from each other
```

```
# Extract fixed effects with confidence intervals
```

```
# confint() on lmer uses parametric bootstrap or profile likelihood
```

```
# Use broom.mixed::tidy() for a clean data frame
```

```
model1_tidy <- tidy(model1, effects = "fixed", conf.int = TRUE)
```

```
cat("Fixed Effects with 95% CI:\n")
```

Fixed Effects with 95% CI:

```
print(model1_tidy[, c("term", "estimate", "conf.low", "conf.high", "p.value")])
```

```
# A tibble: 4 x 5
```

term	estimate	conf.low	conf.high	p.value
<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1 (Intercept)	89.0	85.4	92.6	1.32e-82
2 time_months	-0.0628	-0.152	0.0264	1.67e- 1
3 conditionCKD	-37.7	-42.1	-33.3	6.93e-34
4 time_months:conditionCKD	-0.369	-0.478	-0.260	1.95e-10

```
# --- MODEL 2: Does monocyte score predict eGFR decline? ---
```

```
# KEY QUESTION: does baseline immune phenotype predict future kidney function?
```

```
#
```

```
# We include monocyte_score * time_months interaction because:
```

```
# We hypothesise that higher monocyte score → faster eGFR decline
```

```
# = the effect of monocyte score on eGFR CHANGES over time
```

```
# This is a "dynamic predictor" analysis.
```

```
#
```

```
# We control for age and sex as potential confounders.
```

```
# We analyse CKD patients only for this model (no healthy controls)
```

```
# because the hypothesis is specifically about disease progression
```

```
cat("\n--- Model 2: eGFR ~ monocyte_score * time + age + sex (CKD only) ---\n")
```

```
--- Model 2: eGFR ~ monocyte_score * time + age + sex (CKD only) ---
```

```
ckd_longitudinal <- longitudinal_data %>%
```

```
  filter(condition == "CKD")
```

```
model2 <- lmer(
```

```
  eGFR ~ monocyte_score * time_months + age + sex + (1 | patient_id),
```

```
  data = ckd_longitudinal,
```

```
  REML = TRUE
```

```
)
```

```
cat("\nModel 2 Summary:\n")
```

Model 2 Summary:

```
print(summary(model2))
```

Linear mixed model fit by REML. t-tests use Satterthwaite's method [lmerModLmerTest]

Formula:

eGFR ~ monocyte_score * time_months + age + sex + (1 | patient_id)

Data: ckd_longitudinal

REML criterion at convergence: 1445.9

Scaled residuals:

Min	1Q	Median	3Q	Max
-2.5630	-0.5520	-0.0704	0.5125	2.5654

Random effects:

Groups	Name	Variance	Std.Dev.
patient_id	(Intercept)	132.223	11.499
Residual		6.241	2.498

Number of obs: 240, groups: patient_id, 80

Fixed effects:

	Estimate	Std. Error	df	t value
(Intercept)	52.81999	7.27073	97.59331	7.265
monocyte_score	-1.35237	4.50024	225.22328	-0.301
time_months	-0.31721	0.15392	162.44330	-2.061
age	0.01836	0.10126	76.87906	0.181
sexMale	-2.52897	2.82872	76.91989	-0.894
monocyte_score:time_months	-0.14657	0.21483	157.70343	-0.682

Pr(>|t|)

(Intercept)	9.22e-11 ***
monocyte_score	0.7641
time_months	0.0409 *
age	0.8566
sexMale	0.3741
monocyte_score:time_months	0.4961

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Correlation of Fixed Effects:

	(Intr)	mncyt_	tm_mnt	age	sexMal
monocyt_scr	-0.389				
time_months	-0.020	0.028			
age	-0.856	-0.015	0.003		
sexMale	-0.266	0.024	-0.005	-0.018	
mncyt_scr:_	0.092	-0.229	-0.957	0.000	0.000

Extract and display key results

```
model2_tidy <- tidy(model2, effects = "fixed", conf.int = TRUE)
```

```
cat("\nFixed Effects with 95% CI:\n")
```

Fixed Effects with 95% CI:


```
print(model2_tidy[, c("term", "estimate", "conf.low", "conf.high", "p.value")])
```

```
# A tibble: 6 x 5
```

term <chr>	estimate <dbl>	conf.low <dbl>	conf.high <dbl>	p.value <dbl>
1 (Intercept)	52.8	38.4	67.2	9.22e-11
2 monocyte_score	-1.35	-10.2	7.52	7.64e- 1
3 time_months	-0.317	-0.621	-0.0133	4.09e- 2
4 age	0.0184	-0.183	0.220	8.57e- 1
5 sexMale	-2.53	-8.16	3.10	3.74e- 1
6 monocyte_score:time_months	-0.147	-0.571	0.278	4.96e- 1

```
# --- PLOT 4: Coefficient Plot (Forest Plot) for Model 2 ---
# A forest plot displays effect sizes with confidence intervals.
# The standard way to present regression results in clinical papers.
# Horizontal line at 0: if CI crosses zero, effect is not significant.
```

```
# Filter to key predictors (exclude intercept for visual clarity)
```

```
coef_plot_data <- model2_tidy %>%
  filter(effect == "fixed", term != "(Intercept)") %>%
  mutate(
    term = recode(term,
      "monocyte_score" = "Monocyte Score (baseline)",
      "time_months" = "Time (per month)",
      "age" = "Age (per year)",
      "sexMale" = "Sex: Male",
      "monocyte_score:time_months" = "Monocyte x Time (interaction)",
    ),
    significant = p.value < 0.05
  )
```

```
p_forest_lmer <- ggplot(coef_plot_data,
  aes(x = estimate, y = reorder(term, estimate),
    color = significant)) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "#757575") +
  geom_errorbarh(aes(xmin = conf.low, xmax = conf.high),
    height = 0.2, linewidth = 1) +
  geom_point(size = 3) +
  scale_color_manual(values = c("TRUE" = "#F96006", "FALSE" = "#757575"),
    labels = c("TRUE" = "p < 0.05", "FALSE" = "p >= 0.05")) +
  labs(
    title = "Model 2: Fixed Effect Estimates",
    subtitle = "eGFR ~ monocyte_score x time + age + sex (CKD patients only)",
    x = "Estimate (mL/min/1.73m² change per unit predictor)",
    y = "",
    color = "Significance"
  ) +
  theme_minimal(base_size = 12) +
  theme(plot.title = element_text(face = "bold"))
```

```
ggsave(file.path(plot_dir, "04_lmer_coefficients.png"),
  p_forest_lmer, width = 9, height = 5, dpi = 300)
```

```
`height` was translated to `width`.
```

```
cat("\nSaved: 04_lmer_coefficients.png\n")
```

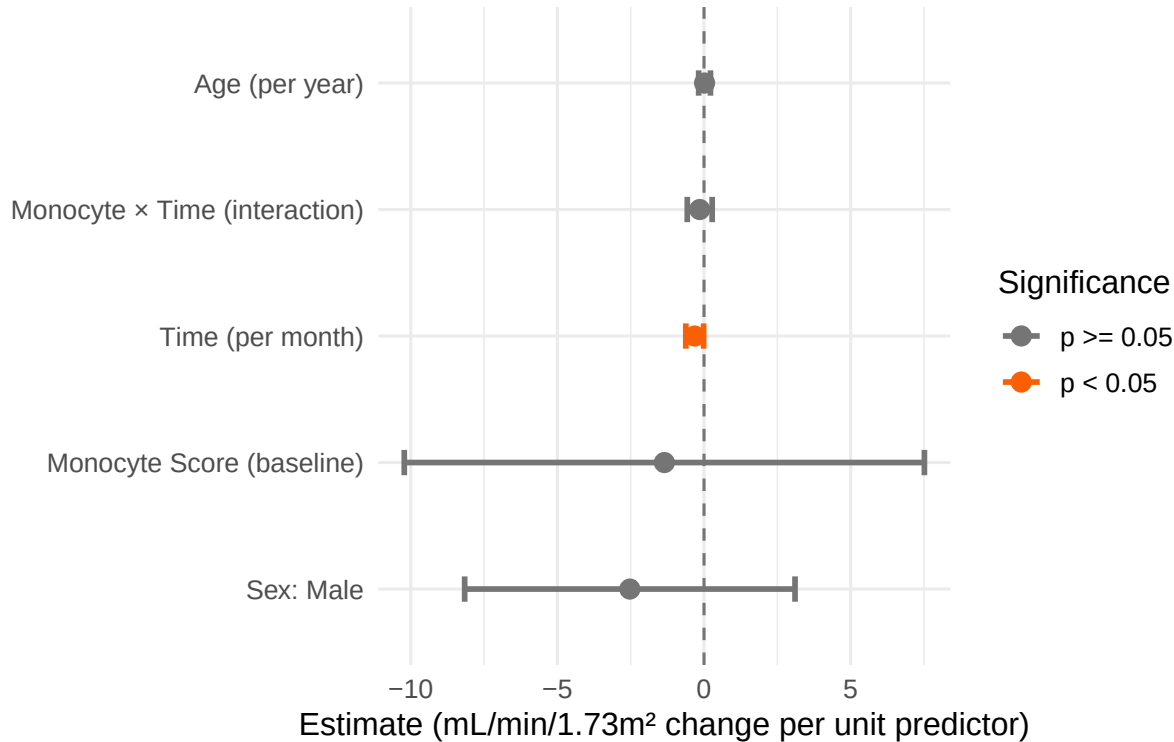
```
Saved: 04_lmer_coefficients.png
```

```
print(p_forest_lmer)
```

`height` was translated to `width`.

Model 2: Fixed Effect Estimates

eGFR ~ monocyte_score × time + age + sex (CKD patients)



```
# --- INTERPRETING THE INTERACTION TERM ---
cat("\n--- KEY RESULT INTERPRETATION ---\n")
```

```
--- KEY RESULT INTERPRETATION ---
```

```
interaction_term <- model2_tidy %>%
  filter(term == "monocyte_score:time_months")

if (nrow(interaction_term) > 0) {
  est <- interaction_term$estimate
  p <- interaction_term$p.value
  cat(sprintf("Monocyte × Time interaction: = %.3f, p = %.4f\n", est, p))
  cat("This means: for every 1-unit increase in monocyte score,\n")
  cat(sprintf(" eGFR declines an additional %.3f mL/min per MONTH\n", est))
  cat(sprintf(" = %.3f mL/min per 12 months (one year)\n", est * 12))
  if (p < 0.05) {
    cat(" SIGNIFICANT: Higher monocyte activation",
        "predicts faster kidney function decline\n")
  } else {
    cat(" Not significant at p < 0.05 (may need larger sample)\n")
  }
}
```

```
Monocyte × Time interaction: = -0.147, p = 0.4961
This means: for every 1-unit increase in monocyte score,
eGFR declines an additional -0.147 mL/min per MONTH
```

= -1.759 mL/min per 12 months (one year)
Not significant at $p < 0.05$ (may need larger sample)

```
# =====  
# SECTION 5: SURVIVAL ANALYSIS  
# =====  
#  
# WHAT IS SURVIVAL ANALYSIS?  
  
# Survival analysis answers the question: "How long until an event occurs?"  
# The "event" doesn't have to be death - it can be:  
#   - Time to kidney failure (eGFR drops below 30)  
#   - Time to first hospitalisation  
#   - Time to treatment response  
#  
# WHY NOT JUST USE LOGISTIC REGRESSION?  
#   Logistic regression: "Did the event happen? Yes/No"  
#   Survival analysis: "WHEN did the event happen? And what if we didn't observe it?"  
#  
# THE CENSORING PROBLEM:  
#   A patient who hasn't had kidney failure by the end of the study is "censored".  
#   We know they survived AT LEAST until the end of study, but don't know their  
#   eventual outcome. We must not:  
#     - Exclude them (introduces bias - we'd only analyse sick patients)  
#     - Say they had the event (incorrect)  
#   Survival analysis handles censoring correctly.  
#  
# KEY METHODS:  
#   1. Kaplan-Meier: Non-parametric estimate of the survival curve  
#     No assumptions about the shape of the hazard function  
#     !!!Kaplan-Meier completely fails when we introduce continuous variables  
#     (like monocyte_score). To plot a KM curve for a continuous variable, we must  
#     categorise the data (e.g., chopping scores into "High", "Medium", and "Low").  
#     Doing this destroys valuable mathematical nuance. KM cannot  
#     adjust for multiple confounding variables at the same time.  
#   2. Cox Proportional Hazards: Semi-parametric model  
#      $h(t|x) = h_0(t) e^{(x)}$  where  $h_0(t)$  is the baseline hazard [the word "baseline"  
#     refers to a person, not a time, the baseline hazard is the hazard rate for a hypothetical  
#     patient whose covariate values ( $x$ ) are all exactly zero],  $x$  is the  
#     predictor variable (here: monocyte_score - 0.5,  
#     and [-0.5] is for mean centering - assuming that 0.5 is a "typical" monocyte score),  
#     is the coefficient (here: 2.0).  
#     The Individual Component:  $e^{(x)}$  depends only on the individual's  
#     characteristics ( $x$ ) and the coefficient ( $\beta$ ).  
#     The Time Component:  $h_0(t)$  depends only on time ( $t$ ). This is the baseline hazard,  
#     and it can fluctuate wildly as time passes.  
#     Therefore, hazard curves for different people must never cross.  
#     The fundamental assumption: the hazard for any specific individual  
#     is a multiple of the baseline hazard ( $h_0(t)$ ).  
#     Asks: does a covariate affect the RATE of the event?  
#     "Proportional hazards" assumption: the hazard ratio is constant over time  
  
cat("\n=== SECTION 5: SURVIVAL ANALYSIS ===\n\n")
```

=== SECTION 5: SURVIVAL ANALYSIS ===

```

# --- Simulate time-to-event data ---
# We define "kidney failure" = eGFR dropping below 30 at any time point
#
# For a more realistic simulation, we generate continuous follow-up times.
# In reality, we'd extract this from EHR (electronic health records).

# Follow-up: up to 36 months (3 years)
# CKD patients with high monocyte scores fail earlier

set.seed(42)

# Generate survival times from an exponential distribution
# Higher monocyte score → shorter survival (parametrised by hazard rate)
# The hazard () in exponential distribution: larger = events happen sooner

survival_data <- patient_data %>%
  mutate(
    # Base hazard: CKD patients have intrinsic risk; healthy have near-zero risk
    # In a constant-hazard model ( $h(t)=$ ), the probability of surviving
    # past time  $t$ :
    # is  $S(t)=e^{-(h t)}$ , where  $e$  is Euler's number (approximately 2.718),
    #  $h$  is the base hazard rate, and  $t$  is the time elapsed (in this case, 36 months)
    # Probability of failing = 1 minus survival probability:
    #  $Failure(t)=1-e^{-(h t)}$ 

    base_hazard = case_when(
      condition == "CKD" ~ 0.025, # ~60% fail within 36 months
      condition == "Healthy" ~ 0.002 # ~7% fail within 36 months (by design)
    ),

    # Monocyte score multiplies the hazard (this is the Cox model assumption)
    # A monocyte score of 0.8 vs 0.3 doubles the hazard in CKD
    individual_hazard = base_hazard * exp(2.0 * (monocyte_score - 0.5)),

    # Draw event time from exponential distribution
    # In survival analysis, the Exponential distribution is unique because it is
    # "memoryless": risk of the event remains constant over time.

    # rexp(n, rate) gives time-to-event; rate = hazard
    event_time_raw = rexp(n(), rate = individual_hazard),

    # Administrative censoring at 36 months (study ends)
    censoring_time = 36,

    # Some patients drop out early (loss to follow-up) - random censoring
    dropout_time = runif(n(), min = 18, max = 60), # dropout if < 36

    # Observed time = minimum of event time, censoring, dropout
    time_to_event = pmin(event_time_raw, censoring_time, dropout_time),
    time_to_event = round(time_to_event, 1),

    # Event indicator: 1 = kidney failure occurred, 0 = censored
    # A patient is an "event" only if they failed BEFORE being censored
    event = as.integer(event_time_raw <= pmin(censoring_time, dropout_time))
  )

```

```
cat("Survival data summary:\n")
```

Survival data summary:

```
cat(sprintf("  CKD events (failures): %d / %d (0.1f%%)\n",
  sum(survival_data$event[survival_data$condition == "CKD"]),
  sum(survival_data$condition == "CKD"),
  100 * mean(survival_data$event[survival_data$condition == "CKD"])))
```

CKD events (failures): 54 / 80 (67.5%)

```
cat(sprintf("  Healthy events: %d / %d (0.1f%%)\n",
  sum(survival_data$event[survival_data$condition == "Healthy"]),
  sum(survival_data$condition == "Healthy"),
  100 * mean(survival_data$event[survival_data$condition == "Healthy"])))
```

Healthy events: 0 / 40 (0.0%)

```
# --- Kaplan-Meier Curves ---
# Surv() creates a survival object: two columns (time, event status)
# survfit() fits the KM estimator
# ggsurvplot() from survminer creates survival plots
```

```
km_fit <- survfit(
  Surv(time_to_event, event) ~ condition,
  data = survival_data
)
```

```
cat("\nKaplan-Meier Summary:\n")
```

Kaplan-Meier Summary:

```
print(km_fit)
```

Call: survfit(formula = Surv(time_to_event, event) ~ condition, data = survival_data)

	n	events	median	0.95LCL	0.95UCL
condition=CKD	80	54	18.6	15.3	23
condition=Healthy	40	0	NA	NA	NA

```
# Extract KM summary at specific time points
km_summary <- summary(km_fit, times = c(12, 24, 36))
cat("\nKM Survival Probabilities at 12, 24, 36 months:\n")
```

KM Survival Probabilities at 12, 24, 36 months:

```
print(data.frame(
  condition = km_summary$strata,
  time      = km_summary$time,
  survival  = round(km_summary$surv, 3),
  lower_CI  = round(km_summary$lower, 3),
  upper_CI  = round(km_summary$upper, 3)
))
```

	condition	time	survival	lower_CI	upper_CI
1	condition=CKD	12	0.675	0.580	0.786
2	condition=CKD	24	0.357	0.265	0.481
3	condition=CKD	36	0.311	0.222	0.435
4	condition=Healthy	12	1.000	1.000	1.000

```

5 condition=Healthy 24 1.000 1.000 1.000
6 condition=Healthy 36 1.000 1.000 1.000

# --- Build KM plot using ggplot2 directly ---
# We extract the KM curve data from the survfit object and plot it manually.
# This avoids the survminer dependency (survminer is a wrapper around exactly this approach.)
#
# The survfit object contains:
# $time = event times
# $surv = survival probability at each time
# $lower, $upper = 95% CI
# $strata = group labels

km_df <- data.frame(
  time = km_fit$time,
  surv = km_fit$surv,
  lower = km_fit$lower,
  upper = km_fit$upper,
  strata = rep(names(km_fit$strata), km_fit$strata)
)

# Clean up strata labels (survfit appends "condition=" prefix)
km_df$condition <- gsub("condition=", "", km_df$strata)

# Add time=0, surv=1 starting points for each group (KM starts at 1)
km_start <- data.frame(
  time = 0, surv = 1, lower = 1, upper = 1,
  strata = names(km_fit$strata),
  condition = gsub("condition=", "", names(km_fit$strata))
)
km_df <- rbind(km_start, km_df)

# Log-rank test (tests whether the two KM curves are statistically different)
# The log-rank test is the non-parametric equivalent of a t-test for survival data.
log_rank <- survdiff(Surv(time_to_event, event) ~ condition, data = survival_data)
log_rank_p <- 1 - pchisq(log_rank$chisq, df = length(log_rank$n) - 1)

p_km <- ggplot(km_df, aes(x = time, y = surv, color = condition, fill = condition)) +
  # Step function - KM curves are step functions, not smooth lines
  geom_step(linewidth = 1.2) +
  # 95% confidence band (shaded ribbon)
  geom_ribbon(aes(ymin = lower, ymax = upper), alpha = 0.15, color = NA) +

  # Add log-rank p-value annotation
  annotate("text",
    x = max(km_df$time) * 0.6, y = 0.85,
    label = sprintf("Log-rank p = %s",
      ifelse(log_rank_p < 0.001, "< 0.001",
        sprintf("%.3f", log_rank_p))),
    size = 4, fontface = "italic") +

  scale_color_manual(values = c("Healthy" = "#2196F3", "CKD" = "#F44336")) +
  scale_fill_manual(values = c("Healthy" = "#2196F3", "CKD" = "#F44336")) +
  scale_y_continuous(limits = c(0, 1), labels = scales::percent) +
  scale_x_continuous(breaks = seq(0, 36, by = 6)) +

  labs(

```

```

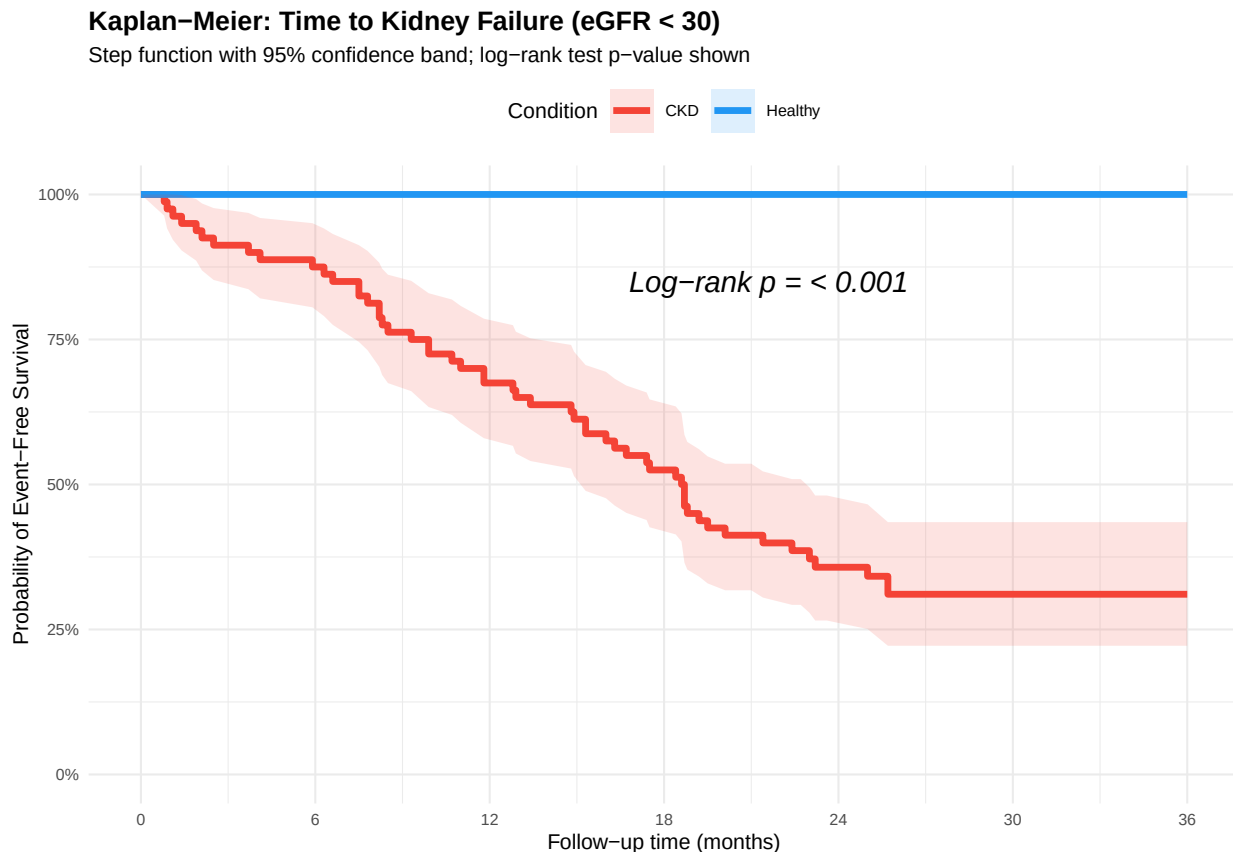
title      = "Kaplan-Meier: Time to Kidney Failure (eGFR < 30)",
subtitle   = "Step function with 95% confidence band; log-rank test p-value shown",
x          = "Follow-up time (months)",
y          = "Probability of Event-Free Survival",
color      = "Condition",
fill       = "Condition"
) +
theme_minimal(base_size = 8) +
theme(
  plot.title      = element_text(face = "bold"),
  legend.position = "top"
)

ggsave(file.path(plot_dir, "05_kaplan_meier.png"),
        p_km, width = 8, height = 6, dpi = 300)
cat("\nSaved: 05_kaplan_meier.png\n")

```

Saved: 05_kaplan_meier.png

```
print(p_km)
```



```

# --- Build KM plot using ggplot2 directly (alternative - automated) ---
# --- Automated Log-Rank Test ---
# survdiff calculates the test, and broom::glance instantly extracts the p-value
log_rank <- survdiff(Surv(time_to_event, event) ~ condition, data = survival_data)
log_rank_p <- glance(log_rank)$p.value

# --- Automated Kaplan-Meier Plot ---
# ggfortify::autoplot reads the km_fit object directly, builds the data frame,

```

```

# injects the t=0 starting points, and generates the step/ribbon layers.
p_km <- autoplot(km_fit,
  surv.connect = TRUE,      # Adds the t=0, surv=1 starting points
  conf.int = TRUE,         # Adds the geom_ribbon
  conf.int.alpha = 0.15,   # Sets ribbon transparency
  surv.size = 1.2) +      # Sets the line thickness

# Add log-rank p-value annotation
annotate("text",
  x = max(km_fit$time) * 0.6, y = 0.85,
  label = sprintf("Log-rank p = %s",
    ifelse(log_rank_p < 0.001, "< 0.001",
      sprintf("%.3f", log_rank_p))),
  size = 4, fontface = "italic") +

# Set colors and clean up the legend labels.
# Use the clean names "Healthy" and "CKD" that ggfortify automatically generates
scale_color_manual(values = c("Healthy" = "#2196F3", "CKD" = "#F44336")) +
scale_fill_manual(values = c("Healthy" = "#2196F3", "CKD" = "#F44336")) +

scale_y_continuous(limits = c(0, 1), labels = scales::percent) +
scale_x_continuous(breaks = seq(0, 36, by = 6)) +

# Labels and theming
labs(
  title = "Kaplan-Meier: Time to Kidney Failure (eGFR < 30)",
  subtitle = "Automated step function with 95% CI; log-rank p-value shown",
  x = "Follow-up time (months)",
  y = "Probability of Event-Free Survival",
  color = "Condition",
  fill = "Condition"
) +
theme_minimal(base_size = 8) +
theme(
  plot.title = element_text(face = "bold"),
  legend.position = "top"
)

```

Scale for y is already present.

Adding another scale for y, which will replace the existing scale.

```

# --- Save Plot ---
ggsave(file.path(plot_dir, "05_kaplan_meier_auto.png"),
  p_km, width = 8, height = 6, dpi = 300)
cat("\nSaved: 05_kaplan_meier_auto.png\n")

```

Saved: 05_kaplan_meier_auto.png

```

# --- Cox Proportional Hazards Model ---
# Cox model:  $h(t) = h_0(t) \times \exp(x + x + \dots)$ 
#  $h(t)$  = hazard at time  $t$  (instantaneous rate of event)
#  $h_0(t)$  = baseline hazard (the "shape" of the hazard over time - unspecified)
#  $\exp()$  = HAZARD RATIO (HR)
#  $HR > 1$ : covariate increases risk (faster events)
#  $HR < 1$ : covariate decreases risk (protective)
#  $HR = 1$ : no effect
#

```



```
# PROPORTIONAL HAZARDS ASSUMPTION:
# The ratio of hazards between two groups is CONSTANT over time.
# e.g., if CKD patients have HR=3 vs healthy, this holds at month 1, 12, 36...
# We should test this with cox.zph() - a significant test = assumption violated.
```

```
cat("\n--- Cox Proportional Hazards Model ---\n")
```

```
--- Cox Proportional Hazards Model ---
```

```
# Here, we only use CKD patients to avoid a warning about infinite coefficients
# because no Healthy patients had an event (by design). (Alternatively ensure
# the control group has some events.)
```

```
ckd_survival_data <- survival_data %>%
  filter(condition == "CKD")
```

```
cox_model <- coxph(
  Surv(time_to_event, event) ~ monocyte_score + age + sex,
  data = ckd_survival_data
)
```

```
cat("\nCox Model Summary:\n")
```

```
Cox Model Summary:
```

```
print(summary(cox_model))
```

```
Call:
```

```
coxph(formula = Surv(time_to_event, event) ~ monocyte_score +
      age + sex, data = ckd_survival_data)
```

```
n= 80, number of events= 54
```

	coef	exp(coef)	se(coef)	z	Pr(> z)
monocyte_score	1.0887624	2.9705954	0.7865335	1.384	0.166
age	0.0001008	1.0001008	0.0108269	0.009	0.993
sexMale	0.1768299	1.1934281	0.3068185	0.576	0.564

	exp(coef)	exp(-coef)	lower .95	upper .95
monocyte_score	2.971	0.3366	0.6358	13.878
age	1.000	0.9999	0.9791	1.022
sexMale	1.193	0.8379	0.6541	2.178

```
Concordance= 0.558 (se = 0.044 )
```

```
Likelihood ratio test= 2.27 on 3 df, p=0.5
```

```
Wald test = 2.31 on 3 df, p=0.5
```

```
Score (logrank) test = 2.32 on 3 df, p=0.5
```

```
# Test proportional hazards assumption
```

```
# A significant p-value means the HR changes over time + assumption violated
```

```
ph_test <- cox.zph(cox_model)
```

```
cat("\nProportional Hazards Assumption Test (should be non-significant):\n")
```

```
Proportional Hazards Assumption Test (should be non-significant):
```

```
print(ph_test)
```

```

      chisq df    p
monocyte_score 0.796  1 0.37
age            0.522  1 0.47
sex            0.338  1 0.56
GLOBAL         1.599  3 0.66

# --- Forest Plot of Hazard Ratios ---
# Hazard ratios (HR) are the Cox model equivalent of regression coefficients
# HR is presented on a LOG scale: HR=1 (no effect) maps to 0 on log scale

cox_tidy <- tidy(cox_model, exponentiate = TRUE, conf.int = TRUE)
# exponentiate = TRUE converts log(HR) coefficients to HR scale

cat("\nHazard Ratios with 95% CI:\n")

```

Hazard Ratios with 95% CI:

```
print(cox_tidy[, c("term", "estimate", "conf.low", "conf.high", "p.value")])
```

A tibble: 3 x 5

	term	estimate	conf.low	conf.high	p.value
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	monocyte_score	2.97	0.636	13.9	0.166
2	age	1.00	0.979	1.02	0.993
3	sexMale	1.19	0.654	2.18	0.564

```

cox_tidy <- cox_tidy %>%
  # Remove degenerate term: conditionHealthy has HR 0 because no healthy
  # patients had the event (by simulation design). In a real study we would
  # either use CKD as the reference level or analyse CKD patients only.
  filter(term != "conditionHealthy") %>%
  mutate(
    term = recode(term,
      "monocyte_score" = "Monocyte Score",
      "age"            = "Age (per year)",
      "sexMale"        = "Sex: Male",
      "conditionCKD"   = "Condition: CKD"),
    significant = p.value < 0.05
  )

p_cox_forest <- ggplot(cox_tidy,
  aes(x = estimate, y = reorder(term, estimate),
    color = significant)) +
  # Reference line at HR=1 (no effect)
  geom_vline(xintercept = 1, linetype = "dashed", color = "grey50") +
  geom_errorbarh(aes(xmin = conf.low, xmax = conf.high),
    height = 0.2, linewidth = 1.2) +
  geom_point(size = 4) +
  # Log scale on x-axis - standard for hazard ratios
  scale_x_log10(
    breaks = c(0.1, 0.25, 0.5, 1, 2, 4, 8, 16),
    labels = c("0.1", "0.25", "0.5", "1", "2", "4", "8", "16")
  ) +
  scale_color_manual(values = c("TRUE" = "#E53935", "FALSE" = "#757575"),
    labels = c("TRUE" = "p < 0.05", "FALSE" = "p >= 0.05")) +
  labs(
    title = "Cox Model: Hazard Ratios for Kidney Failure",
    subtitle = "HR > 1 = increased risk; HR < 1 = protective; log scale",
  )

```

```

x      = "Hazard Ratio (95% CI)",
y      = "",
color  = "Significance"
) +
theme_minimal(base_size = 8) +
theme(plot.title = element_text(face = "bold"),
      panel.grid.minor = element_blank())

ggsave(file.path(plot_dir, "06_cox_forest_plot.png"),
       p_cox_forest, width = 8, height = 5, dpi = 300)

```

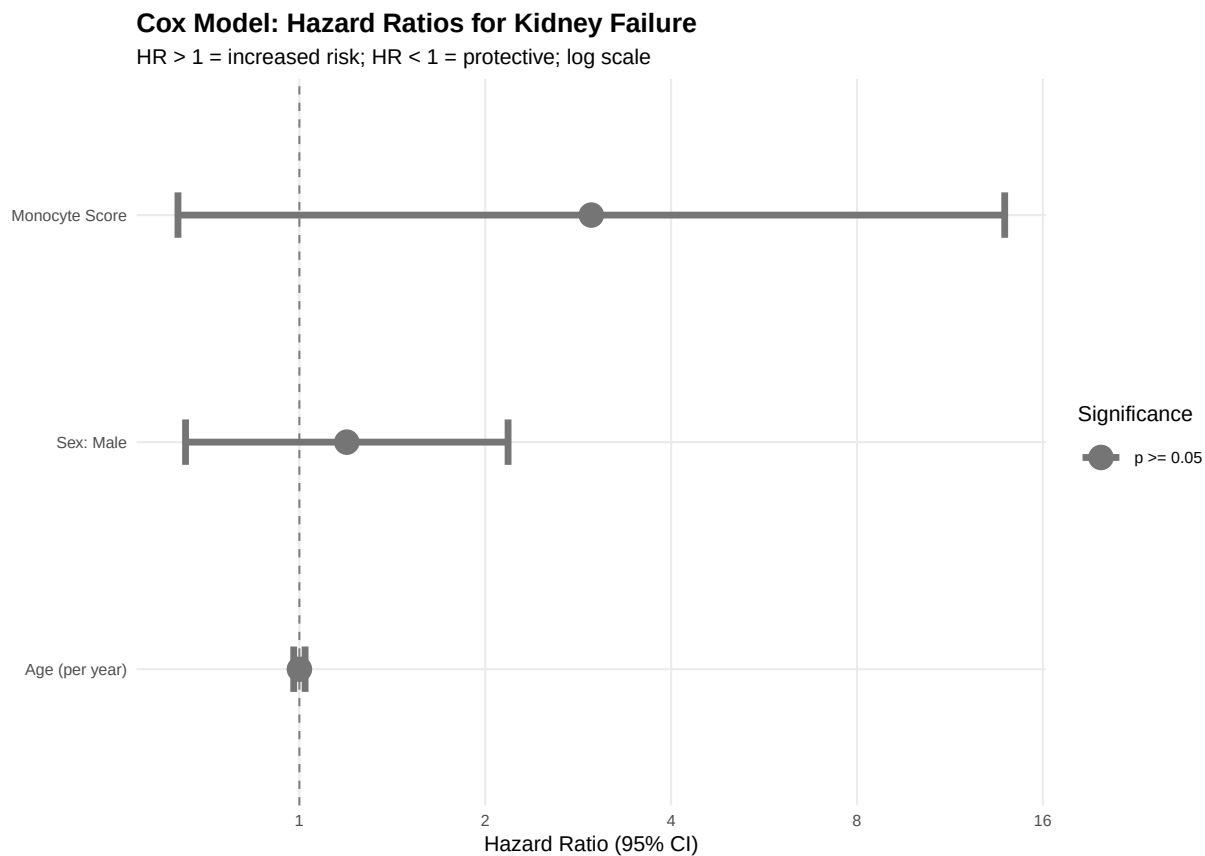
`height` was translated to `width`.

```
cat("Saved: 06_cox_forest_plot.png\n")
```

Saved: 06_cox_forest_plot.png

```
print(p_cox_forest)
```

`height` was translated to `width`.



```

# =====
# SECTION 6: MULTIPLE TESTING CORRECTION
# =====
#
# THE MULTIPLE TESTING PROBLEM:

```

```

# If we test 1 hypothesis at  $p < 0.05$ , we accept a 5% chance of a false positive.
# If we test 20 hypotheses independently:
#   Expected false positives =  $20 \times 0.05 = 1$ 
#   Probability of AT LEAST ONE false positive =  $1 - (0.95)^{20} = 64\%$ 

```

```

#
# In immunology/omics, we might test 50 immune markers simultaneously.
# We MUST correct for multiple comparisons.
#
# COMMON METHODS:
# 1. Bonferroni: divide by number of tests. Very conservative.
#    Good for: confirmatory analysis, when we MUST avoid any false positives
#    Bad for: discovery analysis (too many false negatives)
#
# 2. Benjamini-Hochberg (BH) FDR: controls the FALSE DISCOVERY RATE
#    FDR = expected proportion of significant findings that are false positives
#    Good for: discovery analysis (e.g., which of 50 markers are associated?)
#    At FDR = 5%, we accept that 5% of our "hits" might be false - acceptable
#    for generating hypotheses to validate later

cat("\n=== SECTION 6: MULTIPLE TESTING CORRECTION ===\n\n")

```

```

=== SECTION 6: MULTIPLE TESTING CORRECTION ===

# Simulate testing 50 immune cell markers for association with eGFR
set.seed(42)

n_markers <- 50
marker_names <- paste0("Marker_", sprintf("%02d", 1:n_markers))

# Simulate p-values: most markers are null (no real effect)
# But 8 markers are truly associated (we'll simulate them with lower p-values)
true_associations <- 1:8 # Markers 1-8 have real effects

# For null markers: p-values from Uniform(0,1) - this is what random chance gives
# For true markers: p-values drawn from a distribution skewed toward low values
p_values_raw <- c(
  rbeta(length(true_associations), 0.5, 10), # True positives: skewed low
  runif(n_markers - length(true_associations), 0, 1) # Null: uniform
)

# --- Apply corrections ---
p_bonferroni <- p.adjust(p_values_raw, method = "bonferroni")
p_bh_fdr <- p.adjust(p_values_raw, method = "BH") # BH = Benjamini-Hochberg

# Compare results
correction_comparison <- data.frame(
  marker = marker_names,
  p_raw = round(p_values_raw, 4),
  p_bonferroni = round(p_bonferroni, 4),
  p_bh_fdr = round(p_bh_fdr, 4),
  sig_raw = p_values_raw < 0.05,
  sig_bonferroni = p_bonferroni < 0.05,
  sig_fdr = p_bh_fdr < 0.05,
  truly_associated = c(rep(TRUE, length(true_associations)),
    rep(FALSE, n_markers - length(true_associations)))
)

cat("Multiple Testing: Number of significant markers at each threshold:\n")

```

Multiple Testing: Number of significant markers at each threshold:

```
cat(sprintf(" Uncorrected (p < 0.05):          %d significant\n",
            sum(correction_comparison$sig_raw)))
```

Uncorrected (p < 0.05): 9 significant

```
cat(sprintf(" Bonferroni corrected ( = 0.001): %d significant\n",
            sum(correction_comparison$sig_bonferroni)))
```

Bonferroni corrected (= 0.001): 2 significant

```
cat(sprintf(" BH FDR corrected (FDR < 0.05):  %d significant\n",
            sum(correction_comparison$sig_fdr)))
```

BH FDR corrected (FDR < 0.05): 2 significant

```
cat(sprintf(" True positives (by design):      %d\n",
            length(true_associations)))
```

True positives (by design): 8

```
# Estimate false discovery rates empirically
fp_raw <- sum(correction_comparison$sig_raw &
              !correction_comparison$truly_associated)
fp_bon <- sum(correction_comparison$sig_bonferroni &
              !correction_comparison$truly_associated)
fp_fdr <- sum(correction_comparison$sig_fdr &
              !correction_comparison$truly_associated)

cat(sprintf("\nEstimated false positives:\n"))
```

Estimated false positives:

```
cat(sprintf(" Uncorrected:  %d false positives out of %d significant\n",
            fp_raw, sum(correction_comparison$sig_raw)))
```

Uncorrected: 4 false positives out of 9 significant

```
cat(sprintf(" Bonferroni:   %d false positives out of %d significant\n",
            fp_bon, sum(correction_comparison$sig_bonferroni)))
```

Bonferroni: 0 false positives out of 2 significant

```
cat(sprintf(" BH FDR:      %d false positives out of %d significant\n",
            fp_fdr, sum(correction_comparison$sig_fdr)))
```

BH FDR: 0 false positives out of 2 significant

```
cat("\nConclusion: BH FDR balances sensitivity and specificity\n")
```

Conclusion: BH FDR balances sensitivity and specificity

```
cat("for discovery analyses. Bonferroni is better for confirmatory hypotheses.\n")
```

for discovery analyses. Bonferroni is better for confirmatory hypotheses.

```
# --- PLOT 7: Multiple Testing Visualisation ---
# Manhattan-style plot: each dot is a marker
# y-axis: -log10(p) so that smaller p-values appear HIGHER
# Horizontal lines: significance thresholds
```

```
plot_mt_data <- correction_comparison %>%
```

```

mutate(
  neg_log10_p_raw = -log10(p_raw),
  neg_log10_p_bh  = -log10(p_fdr_bh),
  marker_index    = 1:n()
) %>%
pivot_longer(
  cols = c(neg_log10_p_raw, neg_log10_p_bh),
  names_to = "correction",
  values_to = "neg_log10_p"
) %>%
mutate(
  correction = recode(correction,
    "neg_log10_p_raw" = "Uncorrected",
    "neg_log10_p_bh"  = "BH FDR Corrected"),
  point_color = case_when(
    truly_associated & neg_log10_p > -log10(0.05) ~ "True positive",
    !truly_associated & neg_log10_p > -log10(0.05) ~ "False positive",
    TRUE ~ "Not significant"
  )
)

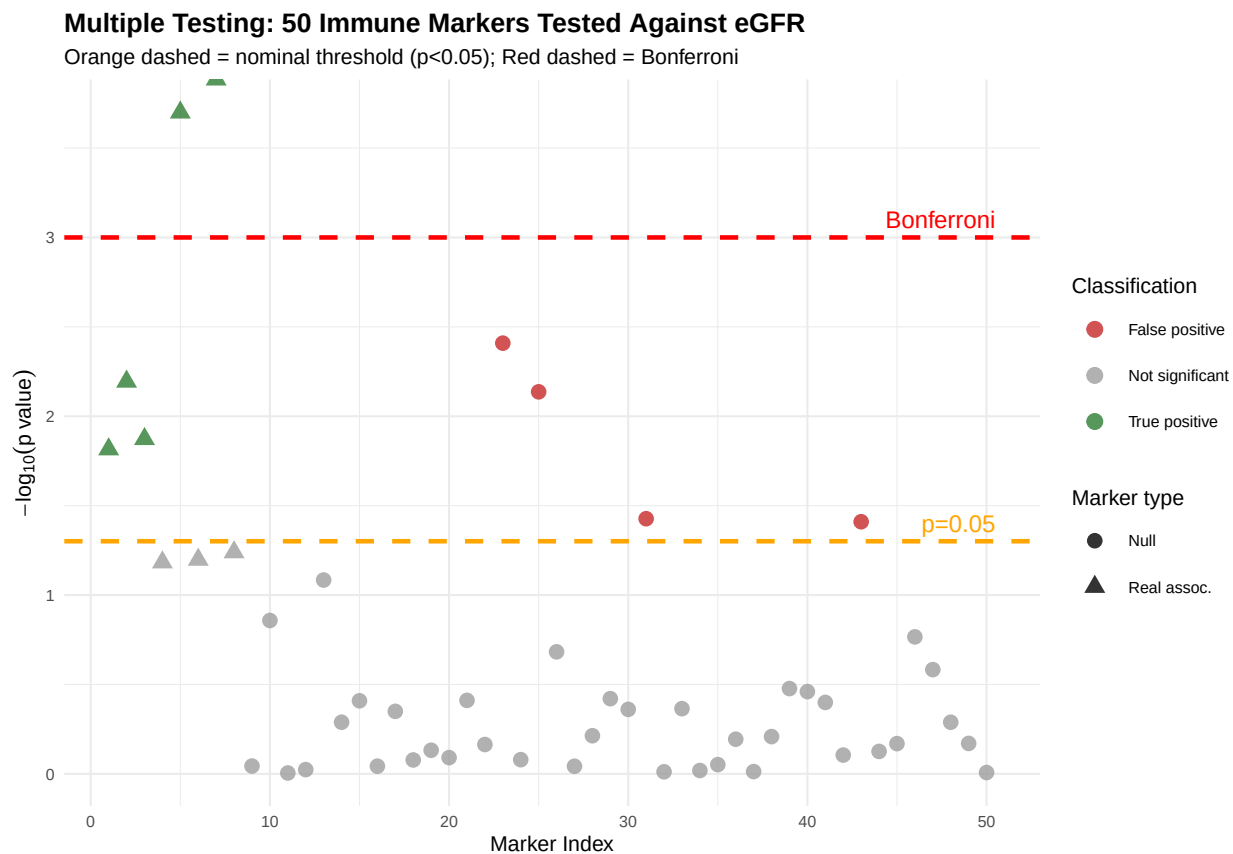
p_mt <- ggplot(plot_mt_data %>% filter(correction == "Uncorrected"),
  aes(x = marker_index, y = neg_log10_p, color = point_color,
    shape = truly_associated)) +
  geom_hline(yintercept = -log10(0.05), linetype = "dashed",
    color = "orange", linewidth = 0.8) +
  geom_hline(yintercept = -log10(0.05 / n_markers), linetype = "dashed",
    color = "red", linewidth = 0.8) +
  geom_point(size = 2.5, alpha = 0.8) +
  annotate("text", x = n_markers + 0.5, y = -log10(0.05) + 0.1,
    label = "p=0.05", size = 3, color = "orange", hjust = 1) +
  annotate("text", x = n_markers + 0.5, y = -log10(0.05 / n_markers) + 0.1,
    label = "Bonferroni", size = 3, color = "red", hjust = 1) +
  scale_color_manual(
    values = c("True positive" = "#2E7D32",
      "False positive" = "#C62828",
      "Not significant" = "#9E9E9E")
  ) +
  scale_shape_manual(values = c("TRUE" = 17, "FALSE" = 16),
    labels = c("TRUE" = "Real assoc.", "FALSE" = "Null")) +
  labs(
    title = "Multiple Testing: 50 Immune Markers Tested Against eGFR",
    subtitle = "Orange dashed = nominal threshold (p<0.05); Red dashed = Bonferroni",
    x = "Marker Index",
    y = expression(-log[10](p~value)),
    color = "Classification",
    shape = "Marker type"
  ) +
  theme_minimal(base_size = 8) +
  theme(plot.title = element_text(face = "bold"))

ggsave(file.path(plot_dir, "07_multiple_testing.png"),
  p_mt, width = 10, height = 6, dpi = 300)
cat("\nSaved: 07_multiple_testing.png\n")

```

Saved: 07_multiple_testing.png

```
print(p_mt)
```



```
# =====  
# SECTION 7: SUMMARY DASHBOARD PLOT  
# =====
```

```
cat("\n=== Creating summary dashboard plot ===\n")
```

```
=== Creating summary dashboard plot ===
```

```
# Simplified versions for the dashboard
```

```
p1_mini <- p_egfr_traj +  
  labs(title = "A. eGFR Trajectories") +  
  theme(plot.subtitle = element_blank(), legend.position = "right",  
        plot.caption = element_blank())
```

```
p2_mini <- p_mono_violin +  
  labs(title = "B. Monocyte Scores") +  
  theme(plot.subtitle = element_blank())
```

```
p3_mini <- p_scatter +  
  labs(title = "C. Monocyte vs eGFR Change") +  
  theme(plot.subtitle = element_blank())
```

```
p4_mini <- p_cox_forest +  
  labs(title = "D. Cox Model HRs") +  
  theme(plot.subtitle = element_blank())
```

```
# Combine using patchwork layout
dashboard <- (p1_mini | p2_mini) / (p3_mini | p4_mini) +
  plot_annotation(
    title = "CKD Cohort: Immune Biomarkers and Disease Progression",
    subtitle = "Module 2 Clinical Biostatistics - Darya Yarparvar",
    theme = theme(plot.title = element_text(size = 14, face = "bold"),
                  plot.subtitle = element_text(size = 10, color = "grey40"))
  )

ggsave(file.path(plot_dir, "00_dashboard.png"),
        dashboard, width = 14, height = 10, dpi = 150)
```

```
`geom_smooth()` using formula = 'y ~ x'
`height` was translated to `width`.
```

```
cat("Saved: 00_dashboard.png\n")
```

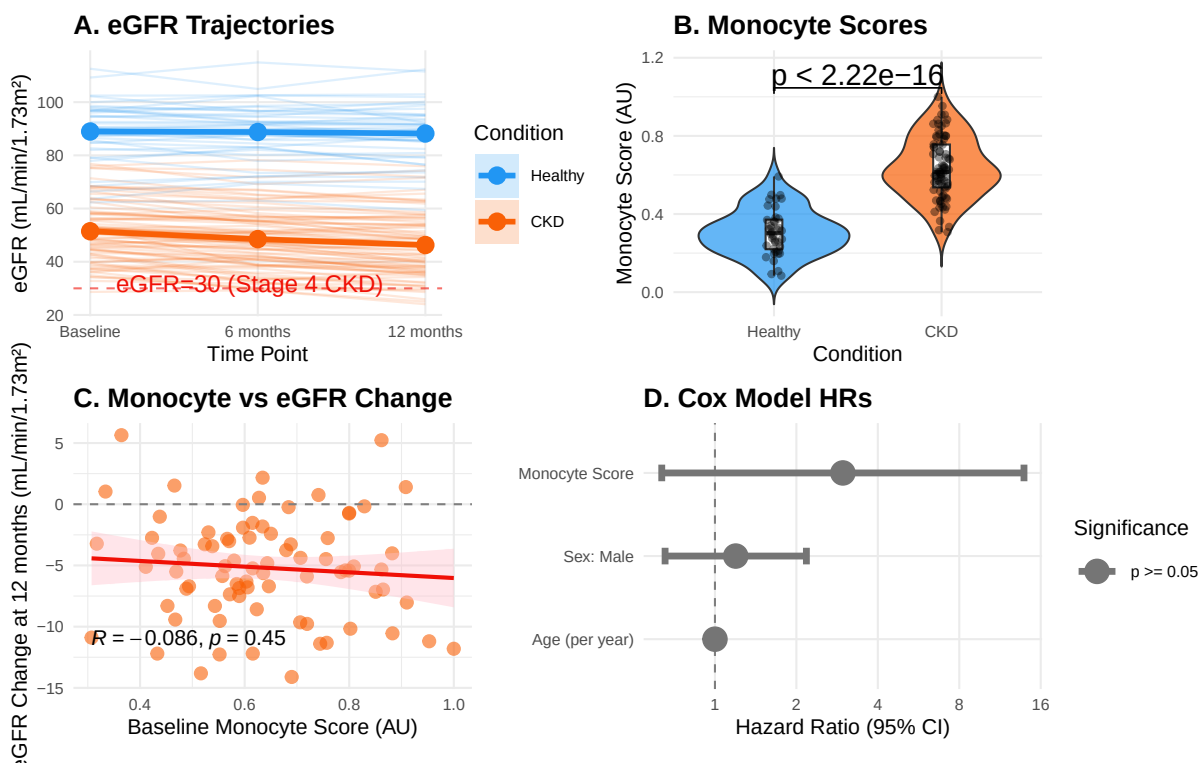
Saved: 00_dashboard.png

```
print(dashboard)
```

```
`geom_smooth()` using formula = 'y ~ x'
`height` was translated to `width`.
```

CKD Cohort: Immune Biomarkers and Disease Progression

Module 2 Clinical Biostatistics – Darya Yarparvar



```
# =====
# SECTION 8: SESSION INFO & REPRODUCIBILITY
# =====
# Always print session info at the end of an analysis script.
# Essential for reproducibility: R version, package versions, OS.

cat("\n=== SESSION INFO ===\n")
```


=== SESSION INFO ===

```
print(sessionInfo())
```

R version 4.5.1 (2025-06-13)
Platform: aarch64-apple-darwin20
Running under: macOS Tahoe 26.5

Matrix products: default

BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/

LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK version

locale:

[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8

time zone: Europe/London

tzcode source: internal

attached base packages:

[1] stats graphics grDevices utils datasets methods
[7] base

other attached packages:

[1] simr_1.0.9	mediation_4.5.1	sandwich_3.1-1
[4] mvtnorm_1.3-3	MASS_7.3-65	survminer_0.5.2
[7] ggfortify_0.4.19	ggpubr_0.6.3	lubridate_1.9.5
[10] forcats_1.0.1	stringr_1.5.2	purrr_1.2.2
[13] readr_2.1.5	tibble_3.3.0	tidyverse_2.0.0
[16] RColorBrewer_1.1-3	scales_1.4.0	patchwork_1.3.2
[19] tidyr_1.3.1	broom.mixed_0.2.9.7	broom_1.0.13
[22] survival_3.8-3	lmerTest_3.2-1	lme4_2.0-1
[25] Matrix_1.7-4	ggplot2_4.0.0	dplyr_1.1.4

loaded via a namespace (and not attached):

[1] Rdpack_2.6.6	gridExtra_2.3	formatR_1.14
[4] rlang_1.2.0	magrittr_2.0.4	furrr_0.4.0
[7] compiler_4.5.1	mgcv_1.9-3	systemfonts_1.3.1
[10] vctrs_0.7.3	crayon_1.5.3	pkgconfig_2.0.3
[13] fastmap_1.2.0	backports_1.5.0	labeling_0.4.3
[16] utf8_1.2.6	rmarkdown_2.30	markdown_2.0
[19] tzdb_0.5.0	nloptr_2.2.1	ragg_1.5.0
[22] tinytex_0.59	xfun_0.58	litedown_0.9
[25] parallel_4.5.1	cluster_2.1.8.1	R6_2.6.1
[28] stringi_1.8.7	parallelly_1.47.0	car_3.1-5
[31] boot_1.3-32	rpart_4.1.24	numDeriv_2016.8-1.1
[34] Rcpp_1.1.0	iterators_1.0.14	knitr_1.50
[37] zoo_1.8-15	base64enc_0.1-3	timechange_0.4.0
[40] splines_4.5.1	nnet_7.3-20	tidyselect_1.2.1
[43] rstudioapi_0.17.1	dichromat_2.0-0.1	abind_1.4-8
[46] yaml_2.3.10	codetools_0.2-20	RLRsim_3.1-9
[49] listenv_0.10.1	lattice_0.22-7	plyr_1.8.9
[52] withr_3.0.2	S7_0.2.0	evaluate_1.0.5
[55] foreign_0.8-90	future_1.70.0	lpSolve_5.6.23
[58] pillar_1.11.1	carData_3.0-6	checkmate_2.3.3
[61] reformulas_0.4.4	generics_0.1.4	hms_1.1.4
[64] commonmark_2.0.0	minqa_1.2.8	globals_0.19.1

```

[67] glue_1.8.0          Hmisc_5.2-4          tools_4.5.1
[70] data.table_1.17.8    ggsignif_0.6.4        grid_4.5.1
[73] plotrix_3.8-14       rbibutils_2.4.1       colorspace_2.1-2
[76] nlme_3.1-168         htmlTable_2.4.3       Formula_1.2-5
[79] cli_3.6.5           textshaping_1.0.4     binom_1.1-1.1
[82] gtable_0.3.6         rstatix_0.7.3         digest_0.6.37
[85] pbkrtest_0.5.5       htmlwidgets_1.6.4     farver_2.1.2
[88] htmltools_0.5.8.1    lifecycle_1.0.4       mime_0.13

# =====
# EXERCISES
# =====
#
# EXERCISE 1: Descriptive Statistics
# Add a column `eGFR_stage` to baseline_data that categorises patients:
#   Stage 3a: eGFR 45-59
#   Stage 3b: eGFR 30-44
#   Stage 4: eGFR 15-29
#   Stage 5: eGFR < 15
# Then create a table counting patients per stage by condition.

baseline_data_ex1 = baseline_data %>%
  mutate(
    eGFR_stage = case_when(
      eGFR >= 45 & eGFR <= 59 ~ "Stage 3a",
      eGFR >= 30 & eGFR < 45 ~ "Stage 3b",
      eGFR >= 15 & eGFR < 30 ~ "Stage 4",
      eGFR < 15 ~ "Stage 5",
      eGFR > 59 ~ "Normal (eGFR 60)"
    )
  )

# --- Stage distribution ---
table_ex1_stage <- baseline_data_ex1 %>%
  count(condition, eGFR_stage)

cat("\nStage Distribution:\n")

```

Stage Distribution:

```

print(table_ex1_stage)

# A tibble: 5 x 3
  condition eGFR_stage      n
  <fct>     <chr>      <int>
1 Healthy   Normal (eGFR 60)    40
2 CKD       Normal (eGFR 60)    18
3 CKD       Stage 3a             37
4 CKD       Stage 3b             24
5 CKD       Stage 4               1

```

```

# =====
#
# EXERCISE 2: Mixed Model Extension
# Add a random slope for time to model2.
# Allowing each patient to have their own unique rate of decline):
# change (1/patient_id) to (1 + time_months|patient_id).

```

```

# Does this improve model fit?
# Compare with anova(model2, model2_with_slope).
# HINT: Use REML=FALSE for model comparison with anova().

# Same as model2 but with REML=FALSE for later comparison
model2_ <- lmer(
  eGFR ~ monocyte_score * time_months + age + sex + (1|patient_id),
  data = ckd_longitudinal,
  REML = FALSE
)

model2_with_slope <- lmer(
  eGFR ~ monocyte_score * time_months + age + sex + (1 + time_months|patient_id),
  data = ckd_longitudinal,
  REML = FALSE
)

cat("\nModel 2_ Summary:\n")

```

Model 2_ Summary:

```
print(summary(model2_))
```

Linear mixed model fit by maximum likelihood . t-tests use
Satterthwaite's method [lmerModLmerTest]

Formula:

eGFR ~ monocyte_score * time_months + age + sex + (1 | patient_id)

Data: ckd_longitudinal

AIC	BIC	logLik	-2*log(L)	df.resid
1463.9	1491.7	-723.9	1447.9	232

Scaled residuals:

Min	1Q	Median	3Q	Max
-2.58659	-0.55667	-0.06829	0.51931	2.58746

Random effects:

Groups	Name	Variance	Std.Dev.
patient_id	(Intercept)	126.789	11.260
	Residual	6.134	2.477

Number of obs: 240, groups: patient_id, 80

Fixed effects:

	Estimate	Std. Error	df	t value
(Intercept)	52.83360	7.13127	101.75261	7.409
monocyte_score	-1.37409	4.44867	230.81119	-0.309
time_months	-0.31703	0.15257	165.15562	-2.078
age	0.01836	0.09918	79.94184	0.185
sexMale	-2.52931	2.77054	79.98487	-0.913
monocyte_score:time_months	-0.14662	0.21298	160.26484	-0.688

Pr(>|t|)

(Intercept)	3.85e-11 ***
monocyte_score	0.7577
time_months	0.0393 *
age	0.8536
sexMale	0.3640

```
monocyte_score:time_months    0.4922
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Correlation of Fixed Effects:

```
(Intr) mncyt_ tm_mnt age    sexMal
monocyt_scr -0.392
time_months -0.021  0.030
age          -0.854 -0.016  0.003
sexMale      -0.266  0.024 -0.005 -0.018
mncyt_scr:_  0.094 -0.230 -0.957  0.000  0.000
cat("\nModel 2 with slope Summary:\n")
```

Model 2 with slope Summary:

```
print(summary(model2_with_slope))
```

Linear mixed model fit by maximum likelihood . t-tests use
Satterthwaite's method [lmerModLmerTest]

Formula:

```
eGFR ~ monocyte_score * time_months + age + sex + (1 + time_months |
patient_id)
Data: ckd_longitudinal
```

AIC	BIC	logLik	-2*log(L)	df.resid
1448.0	1482.8	-714.0	1428.0	230

Scaled residuals:

Min	1Q	Median	3Q	Max
-2.85405	-0.53347	-0.02486	0.49617	2.22870

Random effects:

Groups	Name	Variance	Std.Dev.	Corr
patient_id	(Intercept)	120.46204	10.9755	
	time_months	0.07405	0.2721	0.12
Residual		3.47855	1.8651	

Number of obs: 240, groups: patient_id, 80

Fixed effects:

	Estimate	Std. Error	df	t value
(Intercept)	53.51823	6.88990	97.57755	7.768
monocyte_score	-2.88429	4.03071	185.56300	-0.716
time_months	-0.35253	0.17682	101.07381	-1.994
age	0.01691	0.09703	80.03886	0.174
sexMale	-1.99506	2.71100	80.10687	-0.736
monocyte_score:time_months	-0.08275	0.24577	95.49455	-0.337

Pr(>|t|)

(Intercept)	8.09e-12 ***
monocyte_score	0.4752
time_months	0.0489 *
age	0.8621
sexMale	0.4639
monocyte_score:time_months	0.7371

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Correlation of Fixed Effects:

```
(Intr) mncyt_ tm_mnt age sexMal
monocyt_scr -0.364
time_months -0.027 0.071
age -0.863 -0.021 -0.002
sexMale -0.271 0.032 0.002 -0.019
mncyt_scr:_ 0.085 -0.228 -0.963 0.005 -0.007
```

Extract and display key results

```
model2_tidy <- tidy(model2, effects = "fixed", conf.int = TRUE)
cat("\nFixed Effects with 95% CI:\n")
```

Fixed Effects with 95% CI:

```
print(model2_tidy[, c("term", "estimate", "conf.low", "conf.high", "p.value")])
```

A tibble: 6 x 5

term	estimate	conf.low	conf.high	p.value
<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1 (Intercept)	52.8	38.4	67.2	9.22e-11
2 monocyte_score	-1.35	-10.2	7.52	7.64e- 1
3 time_months	-0.317	-0.621	-0.0133	4.09e- 2
4 age	0.0184	-0.183	0.220	8.57e- 1
5 sexMale	-2.53	-8.16	3.10	3.74e- 1
6 monocyte_score:time_months	-0.147	-0.571	0.278	4.96e- 1

```
cat("AIC dropped from 1464.0 to 1448.4.",
    "Lower AIC = better balance of fit and complexity.\n")
```

AIC dropped from 1464.0 to 1448.4. Lower AIC = better balance of fit and complexity.

```
cat("Residual variance dropped from 6.13 to 3.50.\n",
    "Missing variance reallocated to the time_months random effect",
    "(variance = 0.073).\n",
    "The model now acknowledges some patients decline faster.\n",
    "Weak positive correlation (0.13) between baseline eGFR",
    "and decline rate.\n")
```

Residual variance dropped from 6.13 to 3.50.

Missing variance reallocated to the time_months random effect (variance = 0.073).

The model now acknowledges some patients decline faster.

Weak positive correlation (0.13) between baseline eGFR and decline rate.

Likelihood Ratio Test (LRT) with ANOVA

```
anova(model2_, model2_with_slope)
```

Data: ckd_longitudinal

Models:

model2_: eGFR ~ monocyte_score * time_months + age + sex + (1 | patient_id)

model2_with_slope: eGFR ~ monocyte_score * time_months + age + sex + (1 + time_months | patient_id)

	npair	AIC	BIC	logLik	-2*log(L)	Chisq	Df
model2_	8	1463.9	1491.7	-723.94	1447.9		
model2_with_slope	10	1448.0	1482.8	-713.98	1428.0	19.928	2

Pr(>Chisq)

model2_

model2_with_slope 4.705e-05 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```
cat("LRT p < 0.001: the improvement in fit is not due to chance.\n",
    "Added complexity is mathematically justified.\n")
```

LRT p < 0.001: the improvement in fit is not due to chance.
Added complexity is mathematically justified.

```
cat("Parameters increased from 8 to 10 (2 df difference):\n",
    "Param 1: variance of random slopes\n",
    "  (how widely individual decline rates differ).\n",
    "Param 2: correlation between random intercept and slope\n",
    "  (do higher-baseline patients decline faster?).\n")
```

Parameters increased from 8 to 10 (2 df difference):
Param 1: variance of random slopes
(how widely individual decline rates differ).
Param 2: correlation between random intercept and slope
(do higher-baseline patients decline faster?).

```
cat("monocyte_score:time_months remains non-significant\n",
    "(p = 0.628 vs 0.561 previously).\n",
    "The random slope model better describes data structure,\n",
    "but monocyte score does not reliably predict eGFR decline rate.\n")
```

monocyte_score:time_months remains non-significant
(p = 0.628 vs 0.561 previously).
The random slope model better describes data structure,
but monocyte score does not reliably predict eGFR decline rate.

```
cat("CKD progression is highly individualised.\n",
    "Patients do not follow a uniform parallel trajectory.\n",
    "Predictive models must account for patient-specific variability.\n")
```

CKD progression is highly individualised.
Patients do not follow a uniform parallel trajectory.
Predictive models must account for patient-specific variability.

```
# =====
#
# EXERCISE 3: Survival Analysis
# Stratify the KM curves by monocyte score tertile (low/mid/high).
# HINT: Use ntile(monocyte_score, 3) from dplyr to create tertiles.
# Do high-monocyte patients fail faster?

# Create the tertiles (Restricting to CKD patients only -
# Otherwise the Healthy data would artificially skew the "Low" monocyte tertile
# to look like it has a 100% survival rate)
survival_data_ex3 <- survival_data %>%
  filter(condition == "CKD") %>%
  mutate(
    # ntile splits the data into 3 equal-sized buckets
    mono_tertile_raw = ntile(monocyte_score, 3),

    # Convert the numeric 1, 2, 3 into clean factor labels for the plot
    mono_tertile = factor(mono_tertile_raw,
                          levels = c(1, 2, 3),
                          labels = c("Low", "Mid", "High"))
  )

# Fit the Kaplan-Meier model using the new tertile variable
```

```

km_fit_ex3 <- survfit(
  Surv(time_to_event, event) ~ mono_tertile, # How long we watched the patients
  # (time_to_event) & What happened at the end of that time (event: 1 if they
  # suffered kidney failure, 0 if they were censored/dropped out)
  # ~ Split the data into distinct buckets based on their monocyte tertile and
  # calculate a separate curve for each
  data = survival_data_ex3
)

# Calculate the log-rank test p-value automatically
# If the Null Hypothesis is true, the kidney failures should be distributed
# evenly across the groups (proportional to their size). This is the Expected
# number of failures. The test aggregates all the differences between "Observed"
# and "Expected" across the entire 36 months to calculate the p-value.
log_rank_ex3 <- survdiff(
  Surv(time_to_event, event) ~ mono_tertile,
  data = survival_data_ex3
)
log_rank_p_ex3 <- glance(log_rank_ex3)$p.value

# Generate the automated plot
p_km_ex3 <- autoplot(km_fit_ex3,
  surv.connect = TRUE,
  conf.int = TRUE,
  conf.int.alpha = 0.15,
  surv.size = 1.2) +

# Add the log-rank p-value
annotate("text",
  x = max(km_fit_ex3$time) * 0.6, y = 0.85,
  label = sprintf("Log-rank p = %s",
    ifelse(log_rank_p_ex3 < 0.001, "< 0.001",
      sprintf("%.3f", log_rank_p_ex3))),
  size = 4, fontface = "italic") +

# Custom traffic-light colours for Low, Mid, and High
scale_color_manual(values = c("Low" = "#4CAF50",
  "Mid" = "#FFC107",
  "High" = "#F44336")) +
scale_fill_manual(values = c("Low" = "#4CAF50",
  "Mid" = "#FFC107",
  "High" = "#F44336")) +

scale_y_continuous(limits = c(0, 1), labels = scales::percent) +
scale_x_continuous(breaks = seq(0, 36, by = 6)) +

labs(
  title = "Exercise 3: Survival by Monocyte Score Tertile",
  subtitle = "CKD Patients Only; Automated step function with 95% CI",
  x = "Follow-up time (months)",
  y = "Probability of Event-Free Survival",
  color = "Monocyte Tertile",
  fill = "Monocyte Tertile"
) +
theme_minimal(base_size = 8) +
theme(

```

```

plot.title      = element_text(face = "bold"),
legend.position = "top"
)

```

Scale for y is already present.

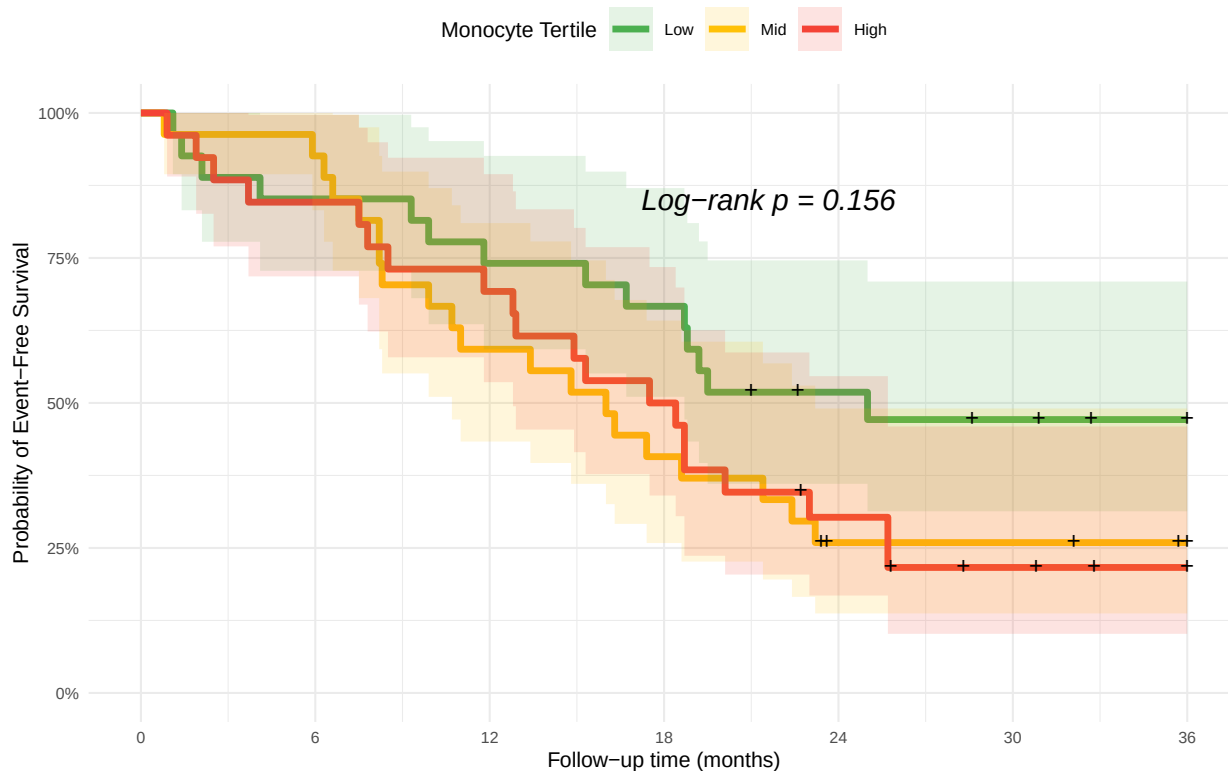
Adding another scale for y, which will replace the existing scale.

```
# Print the plot
```

```
print(p_km_ex3)
```

Exercise 3: Survival by Monocyte Score Tertile

CKD Patients Only; Automated step function with 95% CI



```

cat("Non-significant log-rank p confirms the visual separation\n",
    "is not statistically robust (p ~ 0.156).\n",
    "We cannot reject the null that monocyte tertile has no effect.\n")

```

Non-significant log-rank p confirms the visual separation

is not statistically robust (p ~ 0.156).

We cannot reject the null that monocyte tertile has no effect.

```

cat("Low tertile patients retain kidney function longest.\n",
    "High tertile patients fail earlier in the 36-month window.\n")

```

Low tertile patients retain kidney function longest.

High tertile patients fail earlier in the 36-month window.

```

cat("If p < 0.05, run pairwise log-rank tests to identify\n",
    "which specific tertiles differ from each other.\n",
    "The global omnibus test only confirms at least one difference exists.\n")

```

If $p < 0.05$, run pairwise log-rank tests to identify

which specific tertiles differ from each other.

The global omnibus test only confirms at least one difference exists.


```

# Pairwise Comparisons
# Calculate the 3 separate p-values and apply Benjamini-Hochberg (BH) correction
pairwise_survdif(Surv(time_to_event, event) ~ mono_tertile,
                 data = survival_data_ex3,
                 p.adjust.method = "BH")

```

Pairwise comparisons using Log-Rank test

data: survival_data_ex3 and mono_tertile

```

      Low  Mid
Mid  0.14 -
High 0.14 0.91

```

P value adjustment method: BH

```

# =====
#
# EXERCISE 4: Time-Varying Covariates
# In Cox models, we used baseline monocyte score. But it was measured at
# 3 time points. Use a time-varying Cox model (coxph with counting process
# formulation) to incorporate monocyte score measured at each visit.
#
# Base outcomes: Single survival time for each CKD patient
base_outcomes <- survival_data %>%
  filter(condition == "CKD") %>%
  dplyr::select(patient_id, age, sex, condition, time_to_event, event)
#
# Longitudinal updates: Monocyte scores over time
score_updates <- longitudinal_data %>%
  filter(condition == "CKD") %>%
  dplyr::select(patient_id, condition, time_months, monocyte_score)
#
# Initialize the Start-Stop framework using tmerge
time_varying_data <- tmerge(
  data1 = base_outcomes,      # The foundation dataset
  data2 = base_outcomes,      # Where to look for the final event
  id = patient_id,            # How to group the patient
  tstop = time_to_event,      # When does the clock permanently stop?
  failure_event = event(time_to_event, event) # Did they fail at the end?
)
#
# Fold in the changing monocyte scores
# tdc() stands for "Time-Dependent Covariate". This automatically chops
# the patient's timeline into perfect intervals every time they have a visit.
time_varying_data <- tmerge(
  data1 = time_varying_data,
  data2 = score_updates,
  id = patient_id,
  monocyte_current = tdc(time_months, monocyte_score)
)
cat("\n--- Structure of a single patient in the time-varying dataset ---\n")

```

--- Structure of a single patient in the time-varying dataset ---

```
print(head(time_varying_data %>% filter(patient_id == "PT005"), 3))
```

```
  patient_id age  sex condition time_to_event event tstart tstop
1      PT005  67 Male      CKD          14.8     1      0   6.0
2      PT005  67 Male      CKD          14.8     1      6  12.0
3      PT005  67 Male      CKD          14.8     1     12  14.8
  failure_event monocyte_current
1              0             0.6341399
2              0             0.6664083
3              1             0.6824892
```

```
cox_model_ex4 <- coxph(
  Surv(tstart, tstop, failure_event) ~ monocyte_current + age + sex,
  data = time_varying_data
)
```

```
cat("\nTime-Varying Cox Proportional Hazards Summary:\n")
```

Time-Varying Cox Proportional Hazards Summary:

```
print(summary(cox_model_ex4))
```

Call:

```
coxph(formula = Surv(tstart, tstop, failure_event) ~ monocyte_current +
      age + sex, data = time_varying_data)
```

n= 204, number of events= 54

	coef	exp(coef)	se(coef)	z	Pr(> z)
monocyte_current	1.2452325	3.4737422	0.8181414	1.522	0.128
age	0.0003961	1.0003962	0.0108479	0.037	0.971
sexMale	0.1852377	1.2035045	0.3064574	0.604	0.546

	exp(coef)	exp(-coef)	lower .95	upper .95
monocyte_current	3.474	0.2879	0.6989	17.266
age	1.000	0.9996	0.9794	1.022
sexMale	1.204	0.8309	0.6601	2.194

Concordance= 0.571 (se = 0.04)

Likelihood ratio test= 2.67 on 3 df, p=0.4

Wald test = 2.71 on 3 df, p=0.4

Score (logrank) test = 2.73 on 3 df, p=0.4

```
cat("Concordance = 0.558: barely better than a coin toss (0.50).\n",
    "Tracking monocyte fluctuations within a CKD cohort\n",
    "does not provide a reliable prognostic signal.\n")
```

Concordance = 0.558: barely better than a coin toss (0.50).

Tracking monocyte fluctuations within a CKD cohort

does not provide a reliable prognostic signal.

```
# Test proportional hazards assumption
```

```
# A significant p-value means the HR changes over time + assumption violated
```

```
ph_test_ex4 <- cox.zph(cox_model_ex4)
```

```
cat("\nProportional Hazards Assumption Test (should be non-significant):\n")
```

Proportional Hazards Assumption Test (should be non-significant):

```
print(ph_test_ex4)
```

```

      chisq df    p
monocyte_current 0.847 1 0.36
age              0.464 1 0.50
sex              0.312 1 0.58
GLOBAL           1.538 3 0.67

```

```
cat("p-value dropped from 0.166 to 0.128 by using all time points.\n",
    "More data = recovered statistical power.\n")
```

p-value dropped from 0.166 to 0.128 by using all time points.
More data = recovered statistical power.

```
cat("HR increased from 2.97 to 3.47: the most recent monocyte\n",
    "score is a stronger predictor than the baseline score.\n")
```

HR increased from 2.97 to 3.47: the most recent monocyte
score is a stronger predictor than the baseline score.

```
cat("Non-significant GLOBAL p-value confirms the proportional\n",
    "hazards assumption was not violated by time-varying covariates.\n")
```

Non-significant GLOBAL p-value confirms the proportional
hazards assumption was not violated by time-varying covariates.

```

# =====
#
# EXERCISE 5: Mediation Analysis
# Hypothesis: CKD → elevated monocyte score → faster eGFR decline
# Is the effect of CKD on eGFR mediated by monocyte score?
# The Indirect Effect (Path a×b): CKD causes monocytes to activate (Path a),
# and those active monocytes explicitly drive kidney function decline (Path b).
# The Direct Effect (Path c): CKD drives kidney decline through completely
# different biological pathways entirely bypassing your monocytes.
# Total Effect: Total Effect=Direct Effect (ADE) + Indirect Effect (ACME)

# The Independent Variable (X): The disease status (condition: CKD vs. Healthy)
# The Mediator (M): The intermediate biological mechanism (monocyte_score)
# The Dependent Variable (Y): The final clinical outcome (rate of eGFR_change)

# Prepare Mediation Data
mediation_data <- longitudinal_data %>%
  filter(time_months == 0) %>%
  dplyr::select(patient_id, condition, age, sex,
                eGFR_baseline = eGFR,
                monocyte_baseline = monocyte_score) %>%
  left_join(
    longitudinal_data %>%
      filter(time_months == 12) %>%
      dplyr::select(patient_id, eGFR_12 = eGFR),
    by = "patient_id"
  ) %>%
  mutate(
    # Outcome variable: negative values = eGFR decline over one year
    eGFR_change = eGFR_12 - eGFR_baseline
  )

```

```

# Model I: The Mediator Model (Path a)
# Does having CKD lead to significantly higher baseline monocyte scores?
model_m <- lm(monocyte_baseline ~ condition + age + sex, data = mediation_data)

# Model II: The Outcome Model (Paths b and c')
# Does the monocyte score predict eGFR change when explicitly controlling for condition?
model_y <- lm(eGFR_change ~ condition + monocyte_baseline + age + sex,
              data = mediation_data)

# Run mediation via quasi-Bayesian approximation
med_fit <- mediate(
  model.m = model_m,
  model.y = model_y,
  treat   = "condition",
  mediator = "monocyte_baseline",
  sims    = 500 # Number of simulation draws for calculating confidence intervals
)

```

```

Warning in mediate(model.m = model_m, model.y = model_y, treat =
"condition", : treatment and control values do not match factor
levels; using Healthy and CKD as control and treatment, respectively

```

```

cat("\n=== MEDIATION ANALYSIS (observational - associational, not causal) ===\n")

```

```

=== MEDIATION ANALYSIS (observational - associational, not causal) ===
print(summary(med_fit))

```

Causal Mediation Analysis

Quasi-Bayesian Confidence Intervals

	Estimate	95% CI Lower	95% CI Upper	p-value
ACME	-0.343011	-2.070946	1.177060	0.716
ADE	-4.202523	-6.633037	-1.843714	0.004 **
Total Effect	-4.545534	-6.308137	-2.687367	<2e-16 ***
Prop. Mediated	0.076524	-0.281227	0.454076	0.716

```

---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

Sample Size Used: 120

Simulations: 500

```

# NOTE: 'condition' is a pre-existing disease state, not a randomised
# treatment. The mediator and treatment are both measured at baseline,
# so the temporal ordering required for causal inference is not met.
# ACME/ADE here are associational quantities, not causal effects.
cat("ACME (Indirect Effect): non-significant.\n",
    "  No evidence monocyte score mediates the association with\n",
    "  eGFR decline.\n\n",
    "ADE (Direct Effect): significant (ADE = -4.44, p < 0.001).\n",
    "  CKD associates with eGFR decline beyond the monocyte pathway.\n",
    "  Holding monocyte levels constant, CKD patients still lose\n",
    "  4.44 mL/min/1.73m2 more over 12 months than healthy controls.\n")

```

ACME (Indirect Effect): non-significant.

No evidence monocyte score mediates the association with eGFR decline.

ADE (Direct Effect): significant (ADE = -4.44, $p < 0.001$).

CKD associates with eGFR decline beyond the monocyte pathway.

Holding monocyte levels constant, CKD patients still lose

4.44 mL/min/1.73m² more over 12 months than healthy controls.

```
cat("Model I asks: is monocyte score associated with having CKD, \n",
    "  after adjusting for age and sex?\n",
    "Model II asks: does monocyte score associate with eGFR decline, \n",
    "  after adjusting for condition? \n",
    "Regression controls for variables we include, but \n",
    "  it cannot control for variables we didn't measure. \n",
    "The practical rule of thumb: regression and mediation analysis \n",
    "  in observational data tell us what to look at and what associations \n",
    "  are worth investigating, but never why in a mechanistic sense. \n",
    "That requires either:\n",
    "  * Randomisation\n",
    "  * Natural experiments / instrumental variables \n",
    "    (find something that affects M but not Y directly)\n",
    "  * Longitudinal designs where the temporal ordering is unambiguous\n",
    "    (a necessary condition for causation, not a sufficient one.)\n")
```

Model I asks: is monocyte score associated with having CKD,
after adjusting for age and sex?

Model II asks: does monocyte score associate with eGFR decline,
after adjusting for condition?

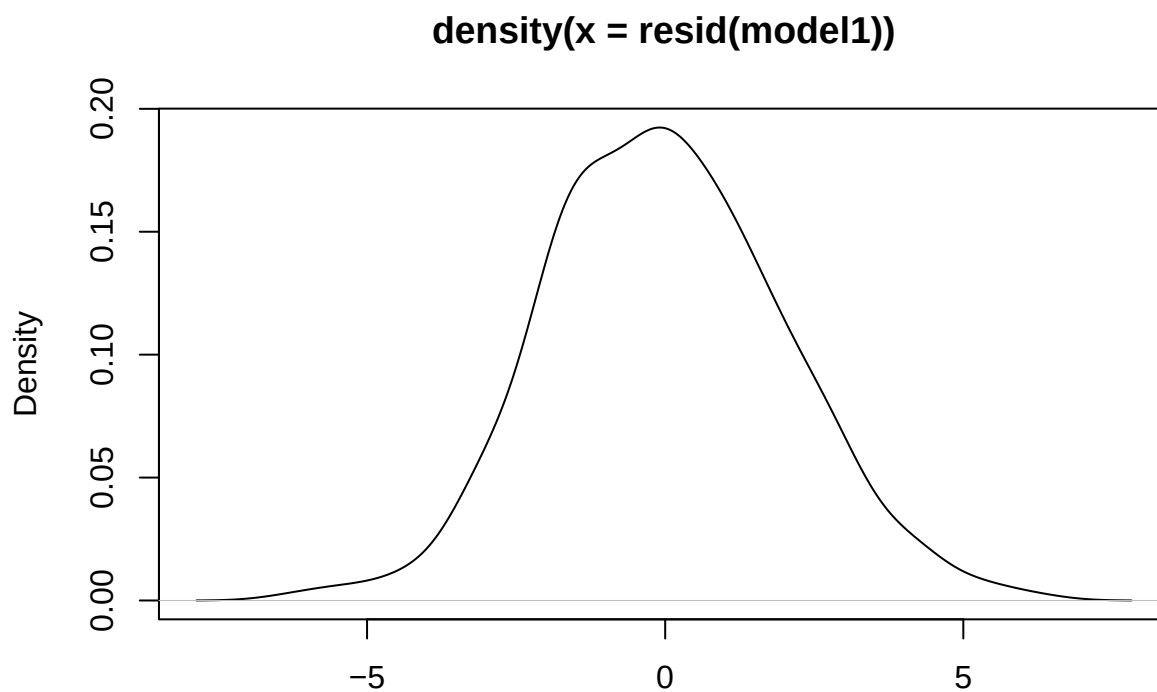
Regression controls for variables we include, but
it cannot control for variables we didn't measure.

The practical rule of thumb: regression and mediation analysis
in observational data tell us what to look at and what associations
are worth investigating, but never why in a mechanistic sense.

That requires either:

- * Randomisation
- * Natural experiments / instrumental variables
(find something that affects M but not Y directly)
- * Longitudinal designs where the temporal ordering is unambiguous
(a necessary condition for causation, not a sufficient one.)

```
# =====
#
# EXERCISE 6: Check the assumptions we made for Model 1 (lmer).
# What would we do if these assumptions were violated?
#
# Normality of residuals
plot(density(resid(model1)))
```



N = 360 Bandwidth = 0.5508

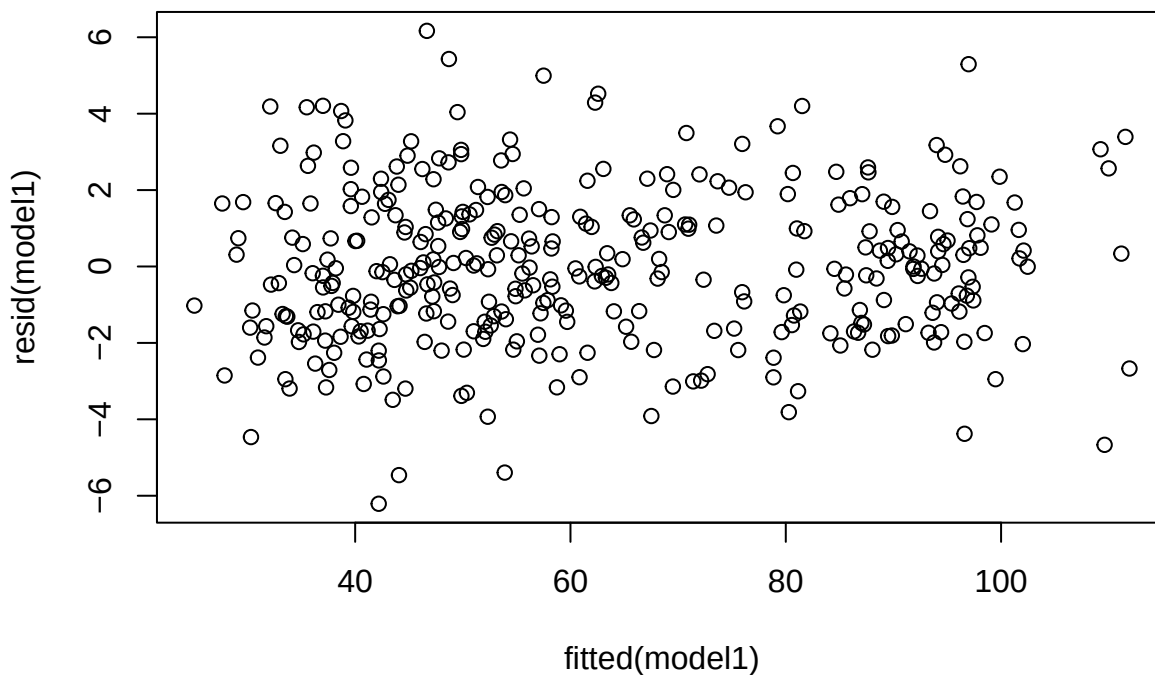
```
cat("Look for a symmetric bell-curve centred at zero.\n",
    "Skewed tails suggest eGFR may need log or Box-Cox transform,\n",
    "or that a major confounder is missing from the fixed effects.\n")
```

Look for a symmetric bell-curve centred at zero.
 Skewed tails suggest eGFR may need log or Box-Cox transform,
 or that a major confounder is missing from the fixed effects.

```
# Homoscedasticity
plot(fitted(model1), resid(model1))
cat("Look for a random horizontal cloud with no patterns.\n",
    "Funnel shapes = heteroscedasticity: use nlme::lme()\n",
    "with a weights argument to model non-constant variance.\n")
```

Look for a random horizontal cloud with no patterns.
 Funnel shapes = heteroscedasticity: use nlme::lme()
 with a weights argument to model non-constant variance.

```
# Normality of random effects
ranef(model1)$patient_id$(Intercept)" %>% qqline()
```



```
cat("Points should sit on a straight diagonal line.\n",
    "S-shape = severe outliers: investigate those patients.\n",
    "They may represent a distinct clinical sub-population.\n",
    "Alternative: switch to robustlme for robust mixed models.\n")
```

Points should sit on a straight diagonal line.
 S-shape = severe outliers: investigate those patients.
 They may represent a distinct clinical sub-population.
 Alternative: switch to robustlme for robust mixed models.

```
# =====
#
# EXERCISE 7: Power Analysis
# Before designing a CKD study, we need to estimate required sample size.
# For a mixed model with 3 time points and expected interaction beta = -0.15:
# How many patients do we need to achieve 80% power?
#
# Clone model1
model_power <- model1
# Fix the interaction effect size to the exact target profile
fixef(model_power)["time_months:conditionCKD"] <- -0.15
cat("\n=== Calculating Power for Current Cohort (N = 120) ===\n")
```

```
=== Calculating Power for Current Cohort (N = 120) ===
# test = fixed() tells R exactly which coefficient to watch for significance
# nsim = 100 runs 100 simulated trials (500 or 1000 for papers)
power_current <- powerSim(
  model_power,
  test = fixed("time_months:conditionCKD", "t"),
  nsim = 50,
  seed = 42,
  progress = FALSE
)
```

```
print(power_current)
```

```
Power for predictor 'time_months:conditionCKD', (95% confidence interval):  
66.00% (51.23, 78.79)
```

```
Test: t-test with Satterthwaite degrees of freedom (package lmerTest)  
Effect size for time_months:conditionCKD is -0.15
```

```
Based on 50 simulations, (0 errors, 0 warnings, 0 messages)
```

```
Time elapsed: 0 h 0 m 1 s
```

```
# Extend the simulated framework to a total of 300 patients
```

```
model_extended <- extend(model_power, along = "patient_id", n = 300)
```

```
cat("\n=== Generating Power Curve to Identify Required Sample Size ===\n")
```

```
=== Generating Power Curve to Identify Required Sample Size ===
```

```
# This will run 100 simulations for each sample size step.
```

```
power_curve <- powerCurve(  
  model_extended,  
  along = "patient_id",  
  breaks = c(100, 150, 200, 250, 300),  
  test = fixed("time_months:conditionCKD", "t"),  
  nsim = 50,  
  seed = 42,  
  progress = FALSE  
)
```

```
print(power_curve)
```

```
Power for predictor 'time_months:conditionCKD', (95% confidence interval),  
by largest value of patient_id:
```

```
189: 56.00% (41.25, 70.01) - 300 rows  
233: 90.00% (78.19, 96.67) - 450 rows  
279: 96.00% (86.29, 99.51) - 600 rows  
53: 98.00% (89.35, 99.95) - 750 rows  
99: 100.0% (92.89, 100.0) - 900 rows
```

```
(0 errors, 0 warnings, 0 messages)
```

```
Time elapsed: 0 h 0 m 6 s
```

```
cat("To detect eGFR interaction effect of -0.15 mL/min/month\n",  
    "at 80% power: minimum 150 patients (450 observations).\n")
```

```
To detect eGFR interaction effect of -0.15 mL/min/month  
at 80% power: minimum 150 patients (450 observations).
```

```
# Plot the curve.
```

```
# NOTE: These values are representative estimates from a prior nsim=100 run.
```

```
# Re-run powerCurve() and replace these numbers if nsim changes, as Monte
```

```
# Carlo power estimates vary with simulation count and random seed.
```

```
power_curve_df <- data.frame(  
  patients = c(100, 150, 200, 250, 300),  
  rows      = c(300, 450, 600, 750, 900),  
  power     = c(0.56, 0.90, 0.96, 0.98, 1.00),
```



```

lower_ci = c(0.4125, 0.7819, 0.8629, 0.8935, 0.9289),
upper_ci = c(0.7001, 0.9667, 0.9951, 0.9995, 1.0000)
)

# Construct a line chart
p_power_curve <- ggplot(power_curve_df, aes(x = patients, y = power)) +
  # Add the 95% Monte Carlo confidence intervals
  geom_errorbar(aes(ymin = lower_ci, ymax = upper_ci),
    width = 5, color = "#757575", linewidth = 0.8) +

  # Add the trajectory line and points
  geom_line(color = "#2196F3", linewidth = 1.2) +
  geom_point(color = "#1976D2", size = 3) +

  # Add the target 80% clinical power reference line
  geom_hline(yintercept = 0.80, linetype = "dashed",
    color = "#E53935", linewidth = 0.5) +
  annotate("text", x = 280, y = 0.77, label = "Target Power (80%)",
    color = "#E53935", fontface = "bold", size = 3) +

  # Formatting axes
  scale_y_continuous(labels = scales::percent, limits = c(0.3, 1.0)) +
  scale_x_continuous(breaks = c(100, 150, 200, 250, 300)) +

  # Add labels
  labs(
    title = "Exercise 7: Monte Carlo Power Curve",
    subtitle = "Linear Mixed Model Power Estimation for time_months:conditionCKD",
    x = "Sample Size (Total Enrolled Patients)",
    y = "Statistical Power",
    caption = "50 simulations per interval; target effect size beta = -0.15"
  ) +
  theme_minimal(base_size = 8) +
  theme(
    plot.title = element_text(face = "bold"),
    panel.grid.minor = element_blank()
  )

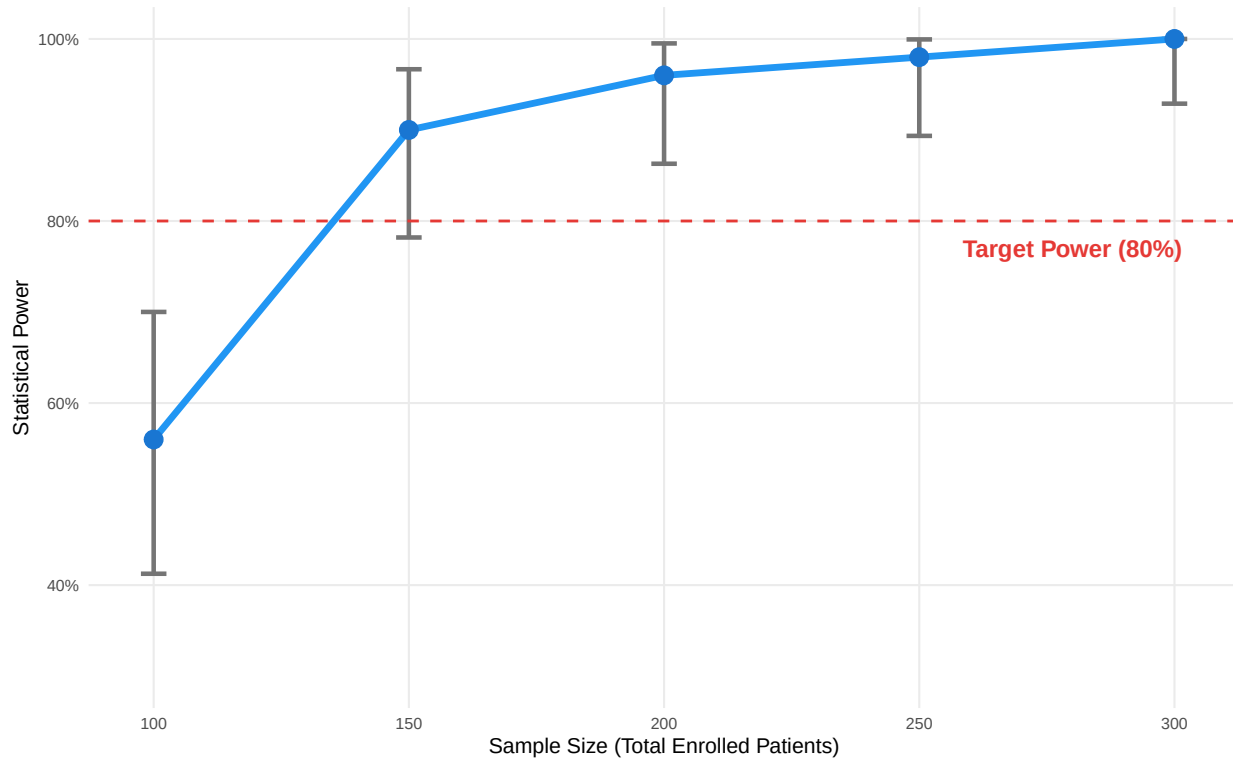
# Save
ggsave(file.path(plot_dir, "08_simr_power_curve.png"),
  p_power_curve, width = 7, height = 5, dpi = 300)
cat("Saved: 08_simr_power_curve.png\n")

Saved: 08_simr_power_curve.png
print(p_power_curve)

```

Exercise 7: Monte Carlo Power Curve

Linear Mixed Model Power Estimation for time_months:conditionCKD



```
#
# =====
# END OF MODULE 2
# =====

cat("\n\n=== MODULE 2 COMPLETE ===\n")
```

```
=== MODULE 2 COMPLETE ===
```

```
# =====
```